

Advanced Deep Learning

1. Convolutional Neural Networks*

Prof. Jianguo Zhang
SUSTech

What's special about images?





WIDTH

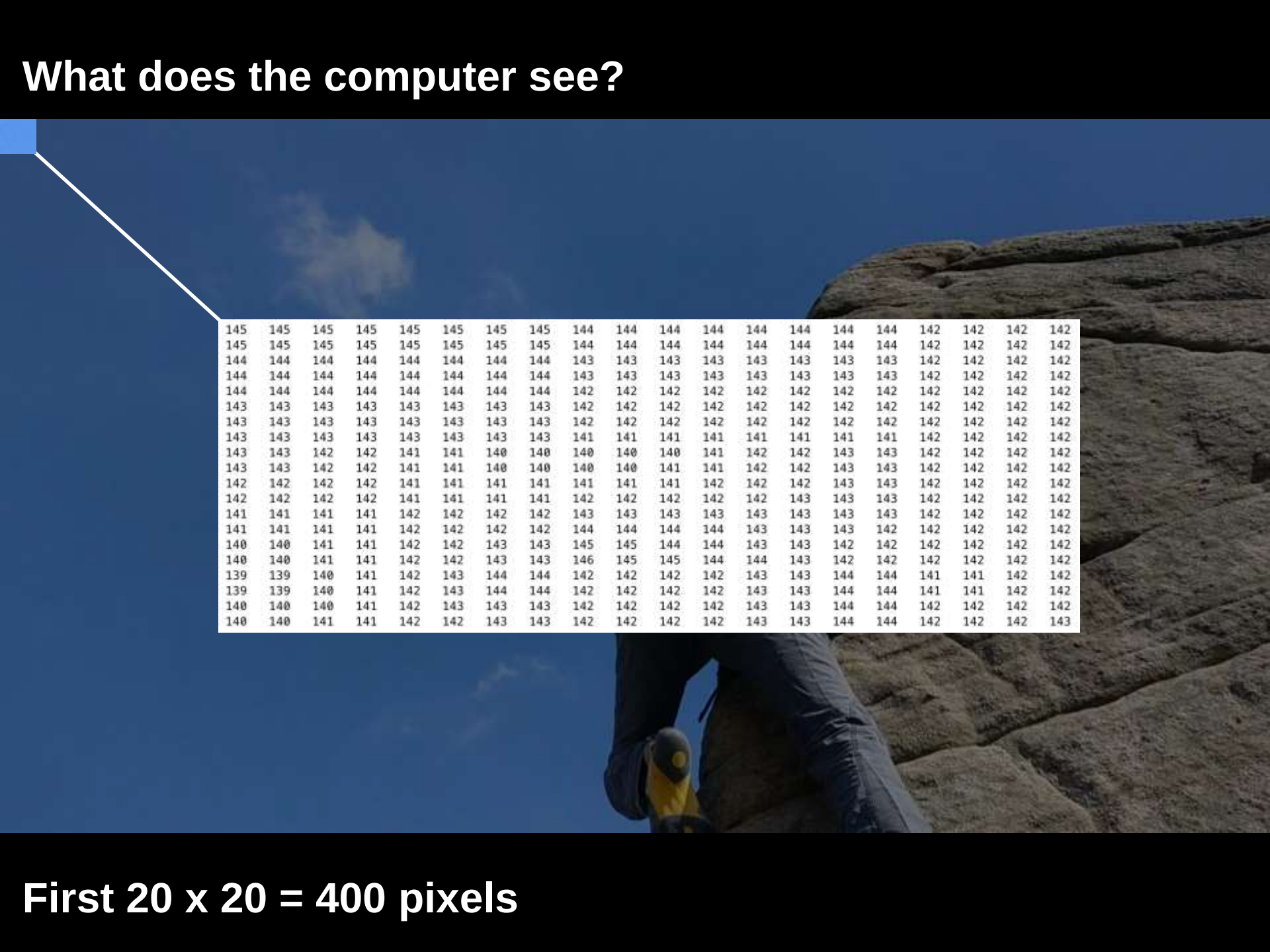


HEIGHT

SPATIAL STRUCTURE



What does the computer see?



A photograph of a person rock climbing on a large, textured rock face. The sky is blue with some clouds. A white line points from the top left corner to a 20x20 grid of numerical values overlaid on the image, representing a 20x20 pixel crop of the scene.

145	145	145	145	145	145	145	145	144	144	144	144	144	144	144	144	142	142	142	142
145	145	145	145	145	145	145	145	144	144	144	144	144	144	144	144	142	142	142	142
144	144	144	144	144	144	144	144	143	143	143	143	143	143	143	143	142	142	142	142
144	144	144	144	144	144	144	144	143	143	143	143	143	143	143	143	142	142	142	142
144	144	144	144	144	144	144	144	142	142	142	142	142	142	142	142	142	142	142	142
143	143	143	143	143	143	143	143	142	142	142	142	142	142	142	142	142	142	142	142
143	143	143	143	143	143	143	143	142	142	142	142	142	142	142	142	142	142	142	142
143	143	143	143	143	143	143	143	141	141	141	141	141	141	141	141	142	142	142	142
143	143	142	142	141	141	140	140	140	140	140	141	142	142	143	143	142	142	142	142
143	143	142	142	141	141	140	140	140	140	141	141	142	142	143	143	142	142	142	142
142	142	142	142	141	141	141	141	141	141	141	142	142	142	143	143	142	142	142	142
142	142	142	142	141	141	141	141	142	142	142	142	142	142	143	143	143	142	142	142
141	141	141	141	142	142	142	142	143	143	143	143	143	143	143	143	142	142	142	142
141	141	141	141	142	142	142	142	144	144	144	144	143	143	143	143	142	142	142	142
140	140	141	141	142	142	143	143	145	145	144	144	143	143	142	142	142	142	142	142
140	140	141	141	142	142	143	143	146	145	145	144	144	143	142	142	142	142	142	142
139	139	140	141	142	143	144	144	142	142	142	142	143	143	144	144	141	141	142	142
139	139	140	141	142	143	144	144	142	142	142	142	143	143	144	144	141	141	142	142
140	140	140	141	142	143	143	143	142	142	142	142	143	143	144	144	142	142	142	142
140	140	141	141	142	142	143	143	142	142	142	142	143	143	144	144	142	142	142	143

First $20 \times 20 = 400$ pixels

1920 (w) x 1080 (h) pixels = 2,073,600 input variables



1920 (w) x 1080 (h) x 3 (RGB) pixels = 6,220,800 input variables



1920 (w) x 1080 (h) x 3 (RGB) pixels = 6,220,800 input variables



What if you feed this to an MLP with 1 hidden layer of 100 units?

1920 (w) x 1080 (h) x 3 (RGB) pixels = 6,220,800 input variables

A low-angle photograph of a person climbing a large, textured rock face. The climber is wearing blue pants and yellow gloves, with one foot visible on a rock ledge. The background is a clear blue sky with a few wispy clouds. A large red rectangular box is superimposed over the center of the image, containing white text.

**600 MILLION
PARAMETERS**

What if you feed this to an MLP with 1 hidden layer of 100 units?

Spot the difference...



Spot the difference...

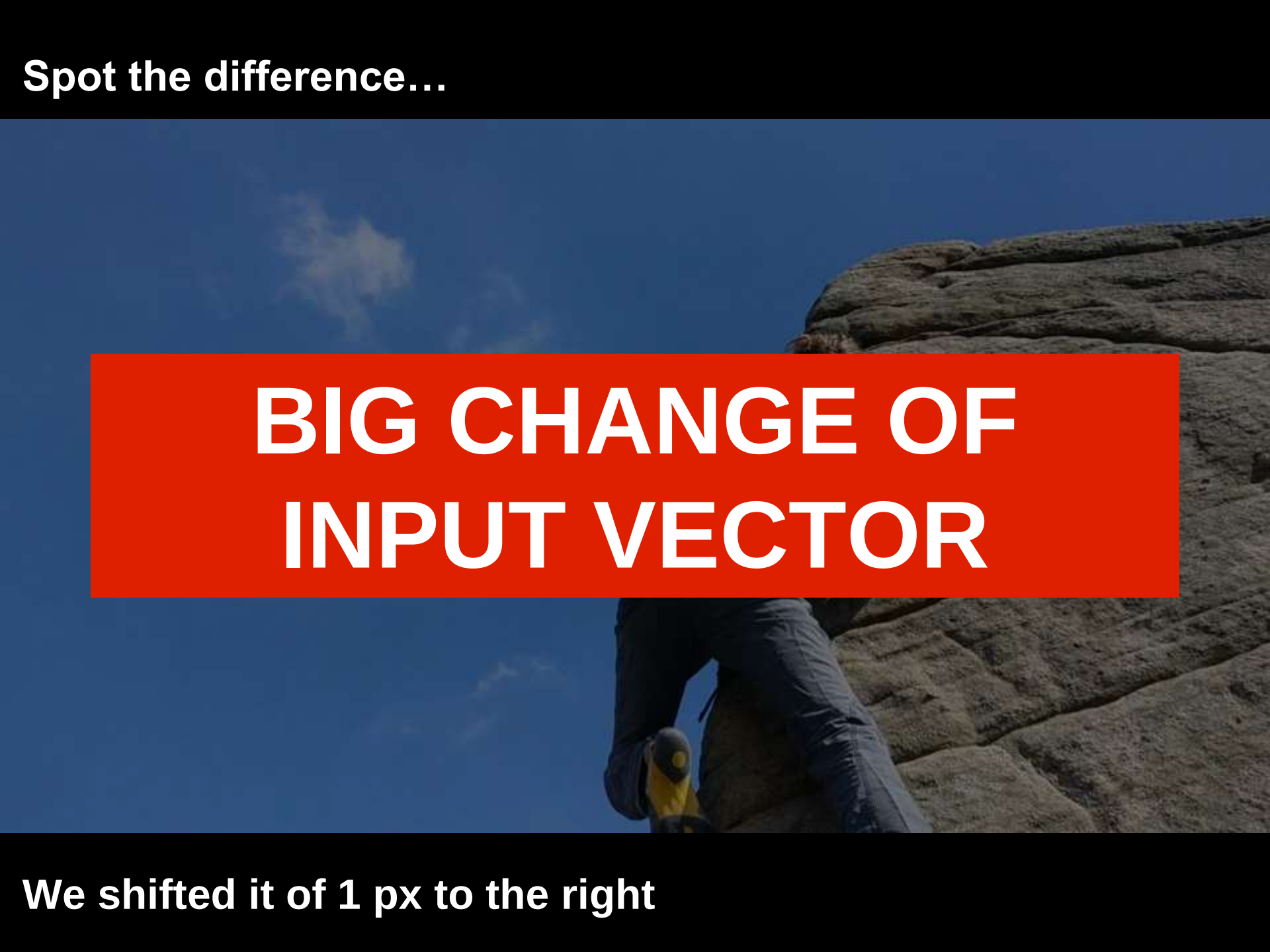


Spot the difference...



We shifted it of 1 px to the right

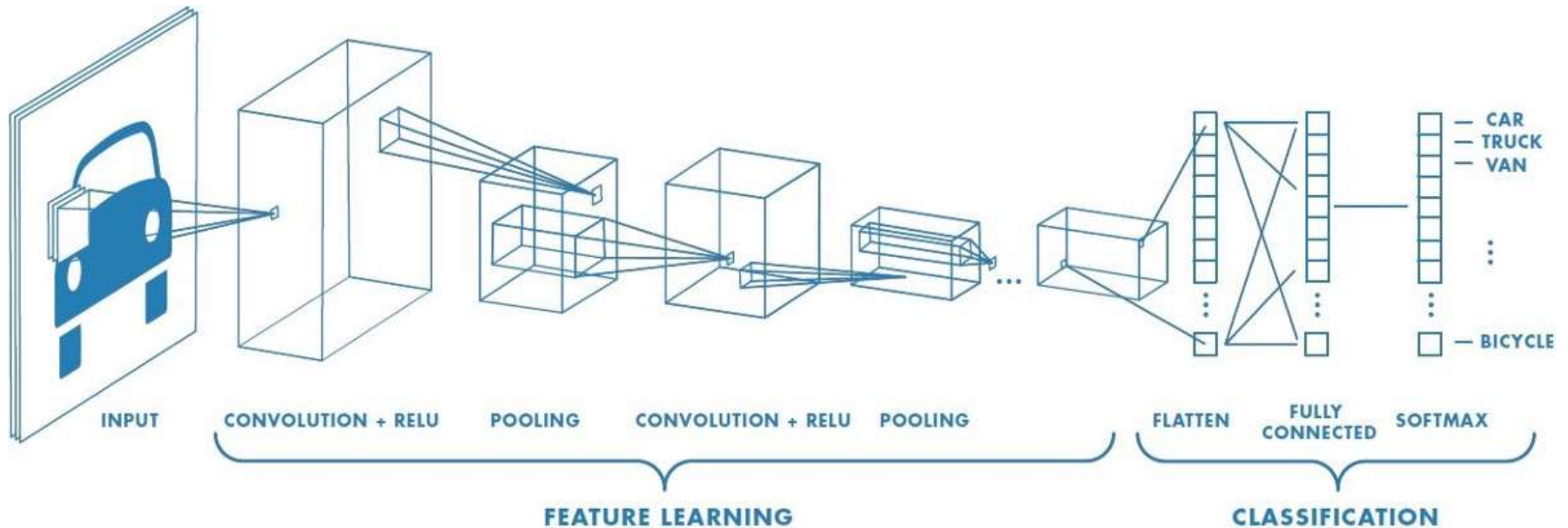
Spot the difference...

A low-angle photograph of a person climbing a large, textured rock face. The climber is wearing blue pants and yellow gloves. The background is a clear blue sky with a few wispy clouds. A large red rectangular box is superimposed over the center of the image, containing white text.

BIG CHANGE OF INPUT VECTOR

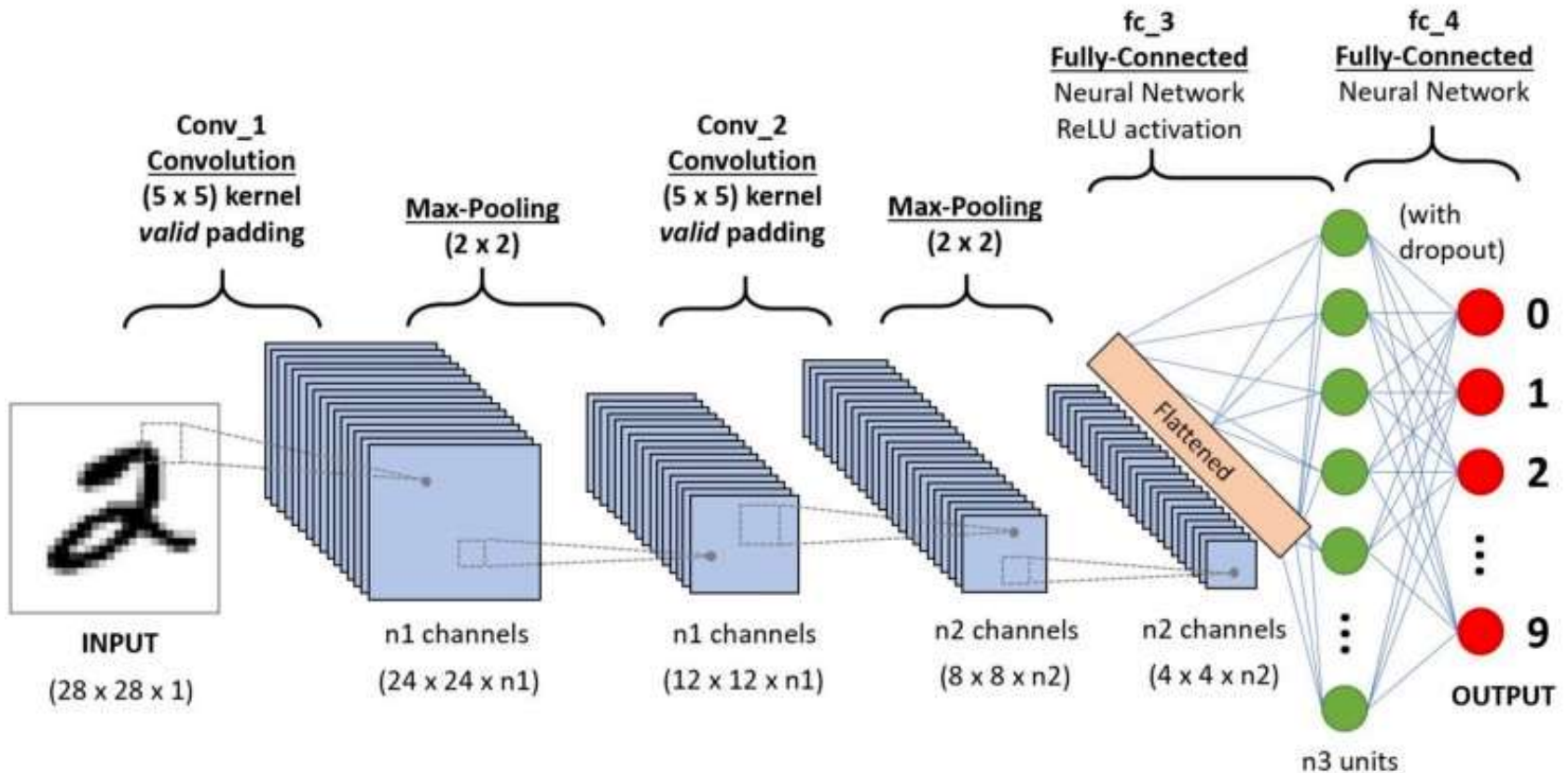
We shifted it of 1 px to the right

Convolutional neural networks



Source: <https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>

Convolutional neural networks



Source: <https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>

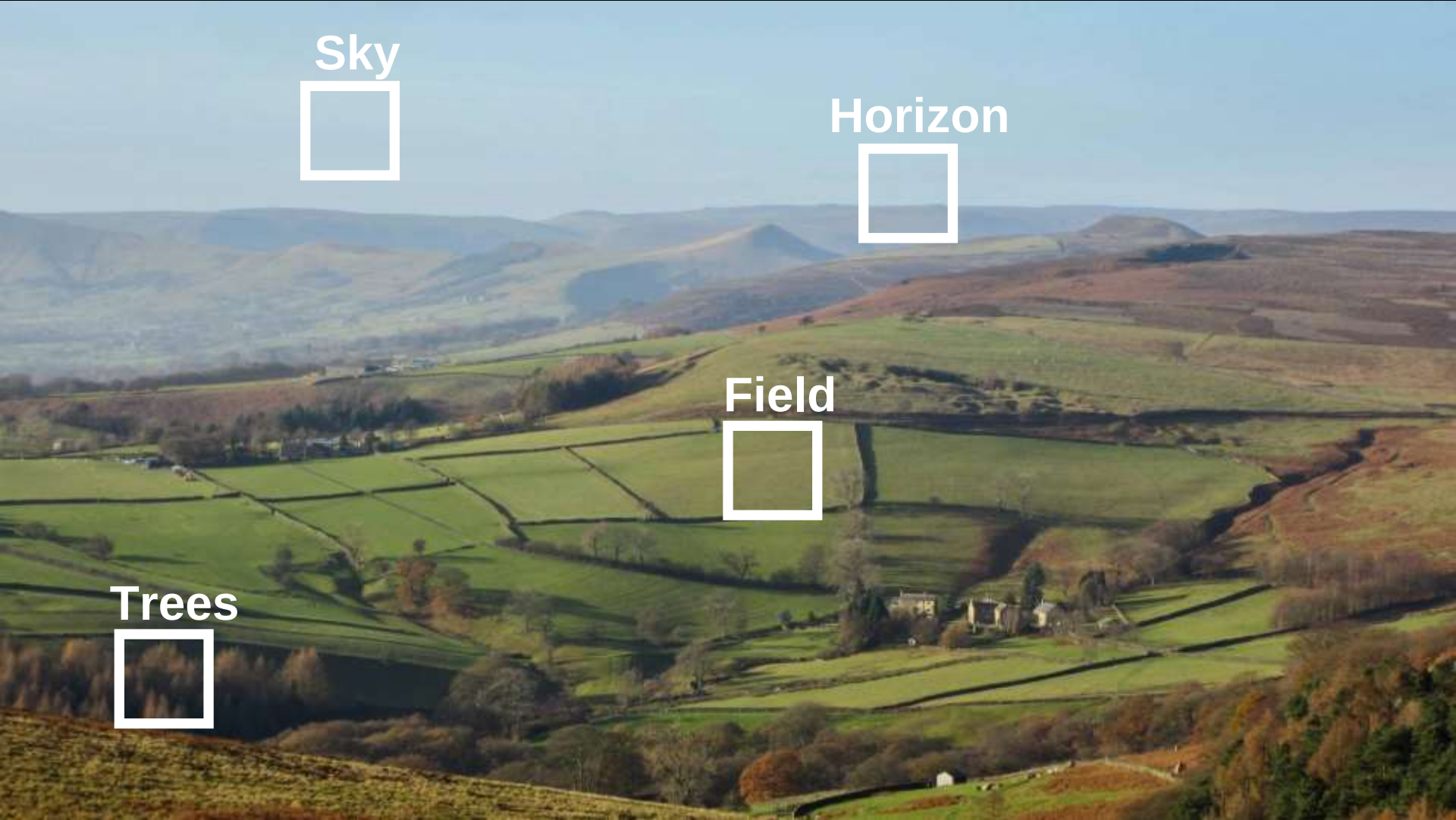
Convolutional neural networks

- Preserve spatial structure by
 - Convolutional filters
- Scale up to process very large inputs (laid out on a grid) using
 - Sparse connections
 - Parameter sharing
- Robust to local variances thanks to
 - Spatial pooling

Neighbouring variables are locally correlated



Neighbouring variables are locally correlated



Key idea of CNNs

- **Replace first layers of matrix multiplications with convolution operations**
- Everything else stays the same
 - Maximum likelihood
 - Back-propagation
 - etc.

2D convolution

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

Source: <https://hackernoon.com/visualizing-parts-of-convolutional-neural-networks-using-keras-and-cats-5cc01b214e59>

Example: Sobel filter

```
%% MATLAB code  
clear  
img = imread('test_image.jpg');  
img = rgb2gray(img);  
imshow(edge(img, 'sobel'));
```



Example: Sobel filter

```
%% MATLAB code  
clear  
img = imread('test_image.jpg');  
img = rgb2gray(img);  
imshow(edge(img, 'sobel'));
```

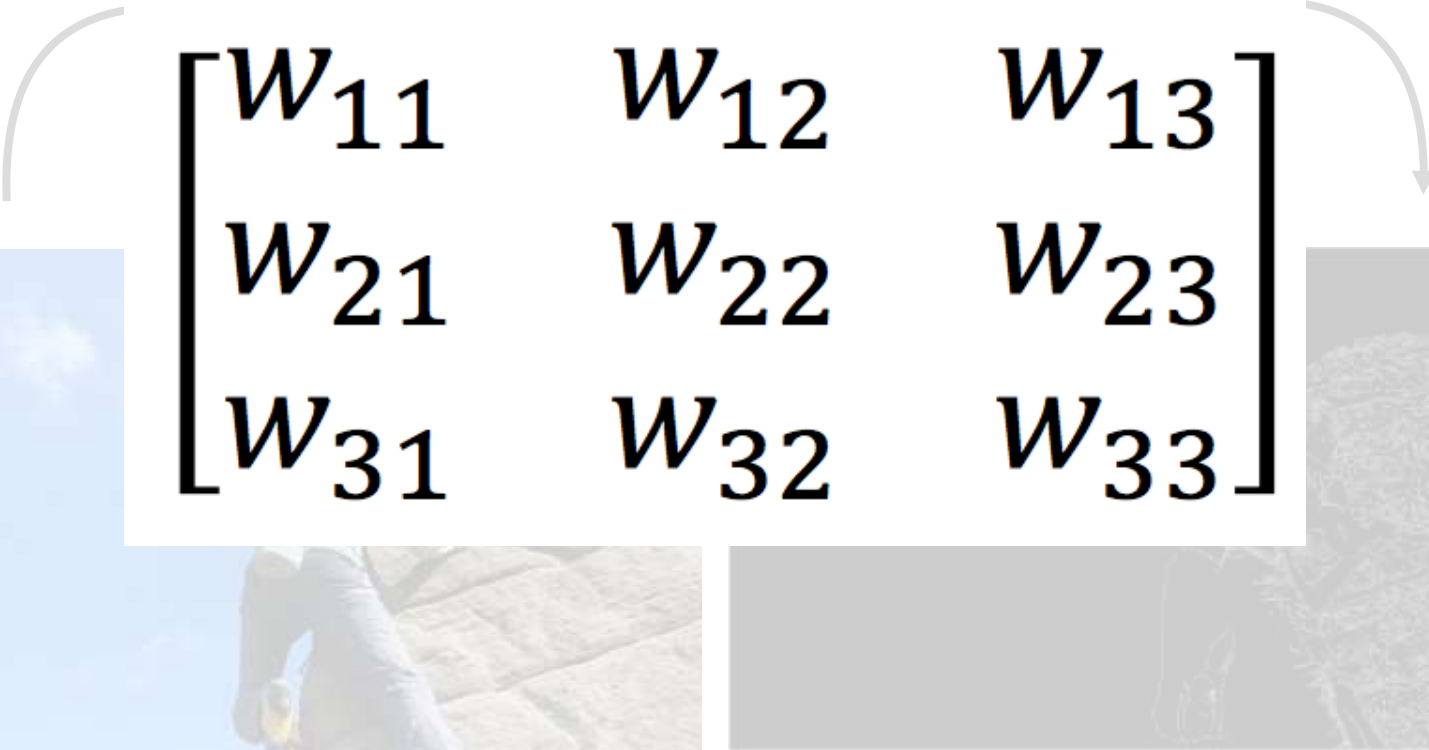


Sobel filter is handcrafted: **can we learn filters from data instead?**

Example: Sobel filter

```
%% MATLAB code
```

```
clear
```

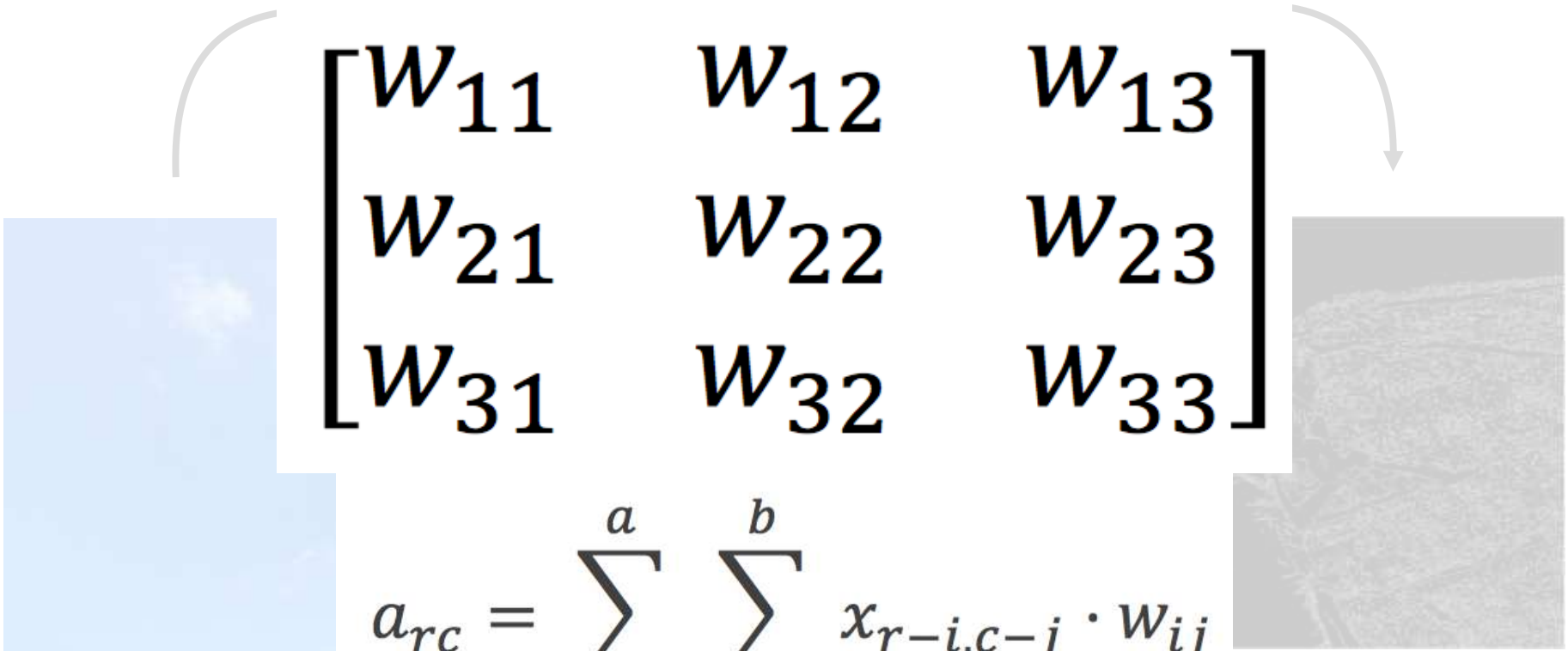

$$\begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

Sobel filter is handcrafted: **can we learn filters from data instead?**

Example: Sobel filter

```
%% MATLAB code
```

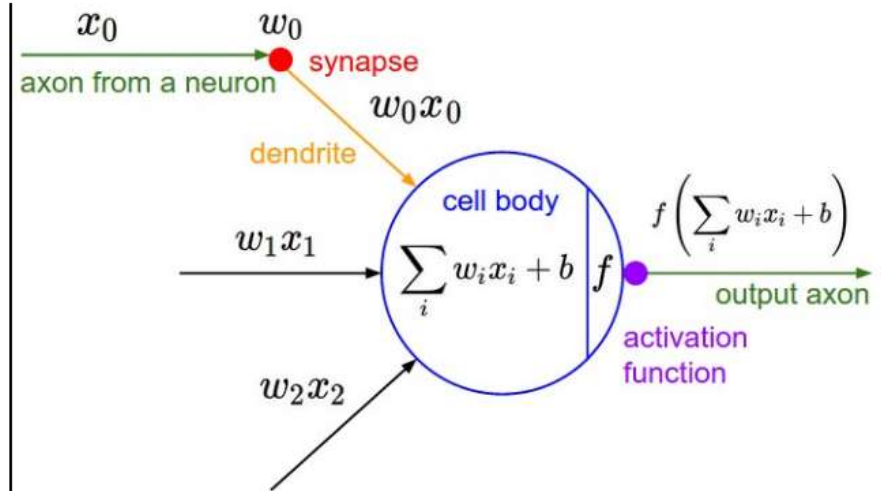
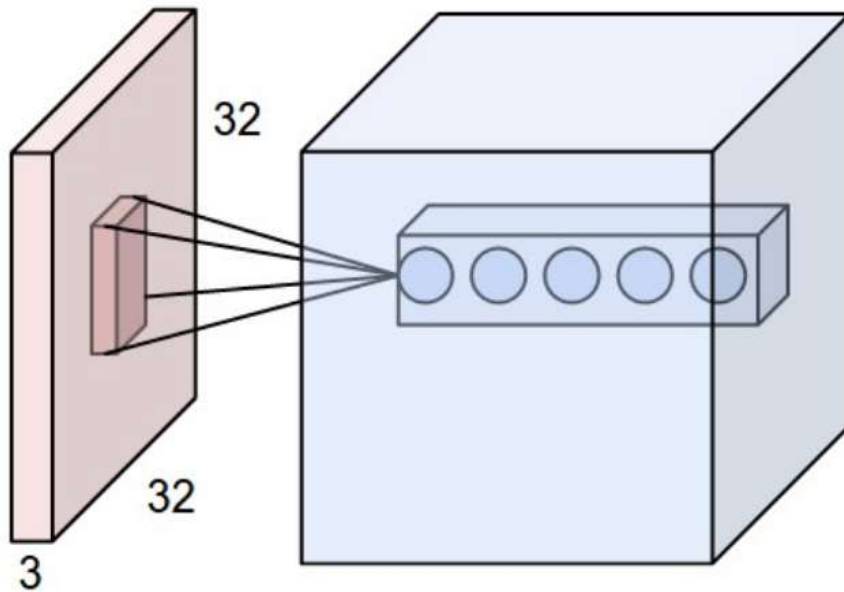
```
clear
```


$$\begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

$$a_{rc} = \sum_{i=-a}^a \sum_{j=-b}^b x_{r-i, c-j} \cdot w_{ij}$$

Convolution is essentially a dot product, just like linear layers

Convolutional layers



Source: <http://cs231n.github.io/convolutional-networks/>

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1



308

+

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2



-498

+

0	1	1
0	1	0
1	-1	1

Kernel Channel #3



164

+ 1 = -25

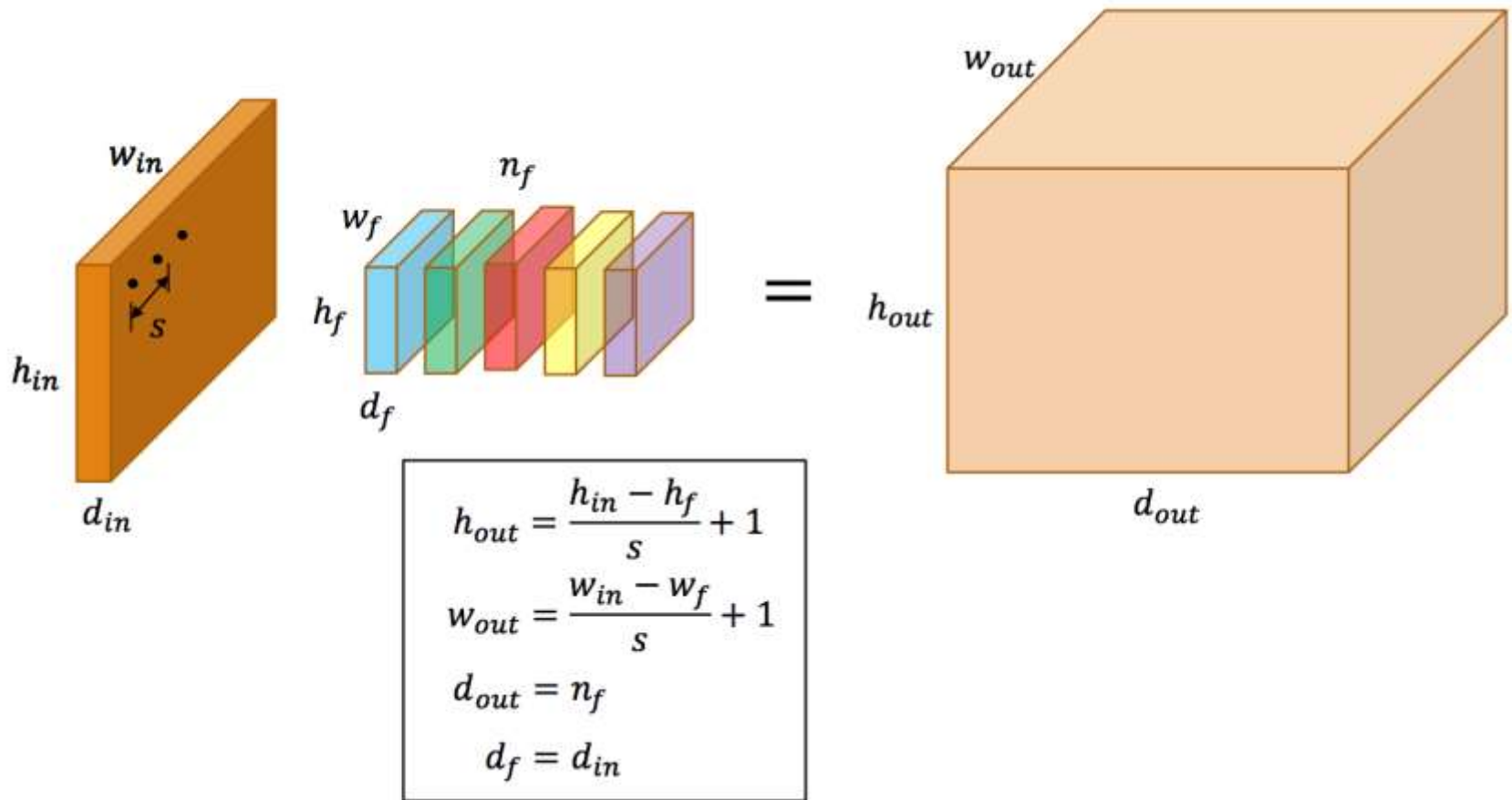


Bias = 1

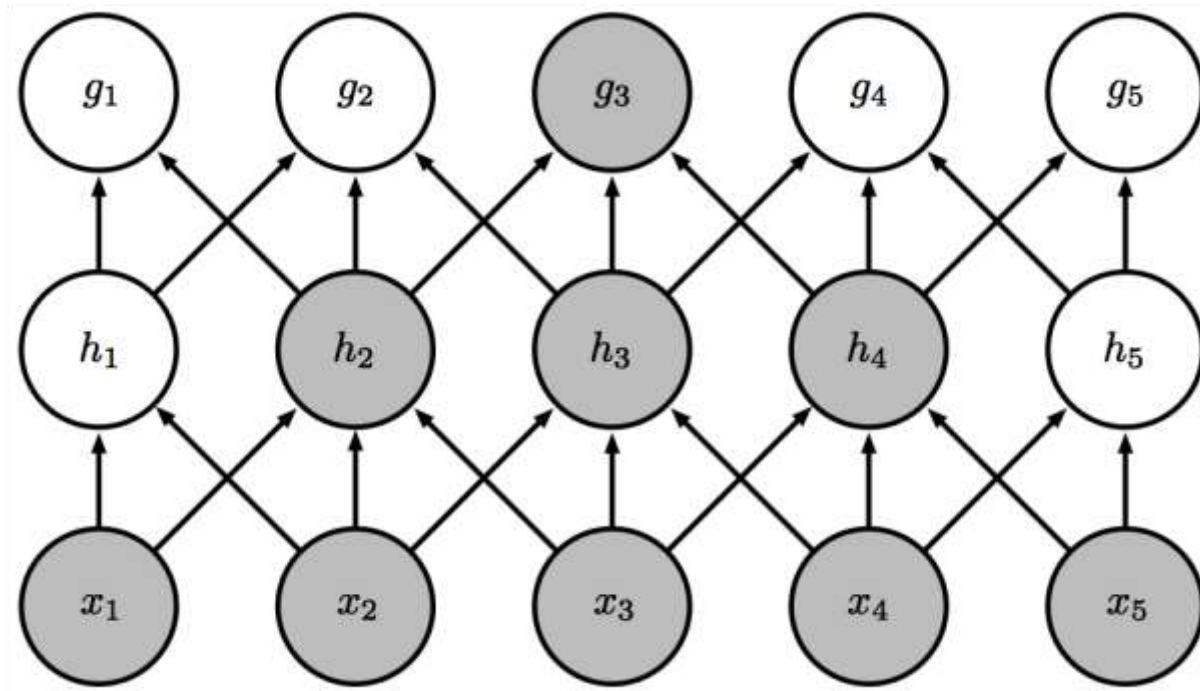
Output

-25				...
				...
				...
				...
...	...	...	...	...

Dimensions after convolution

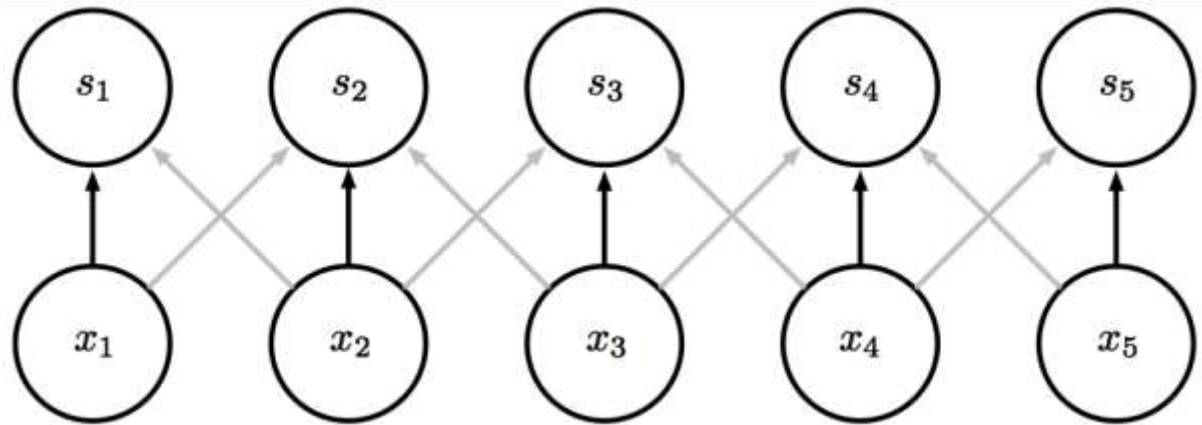


Growing receptive fields

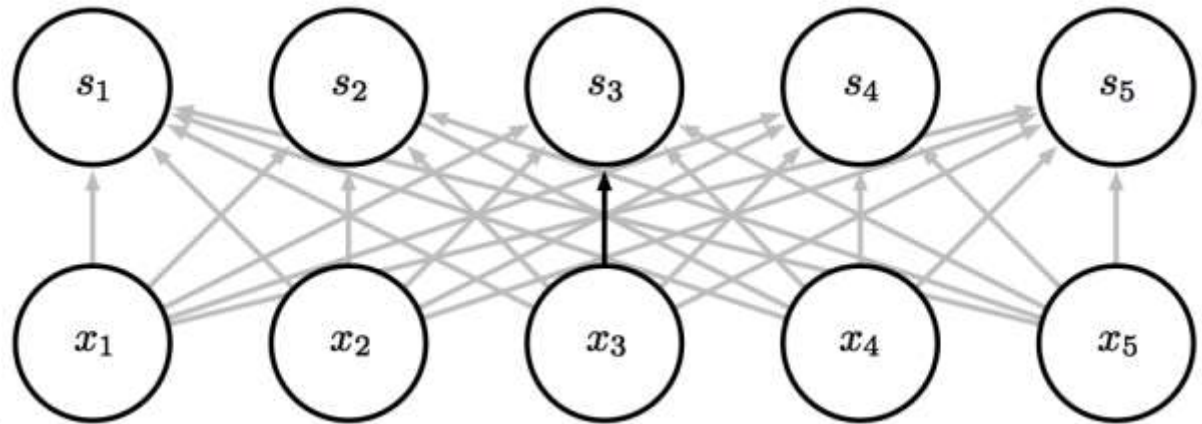


Parameter sharing

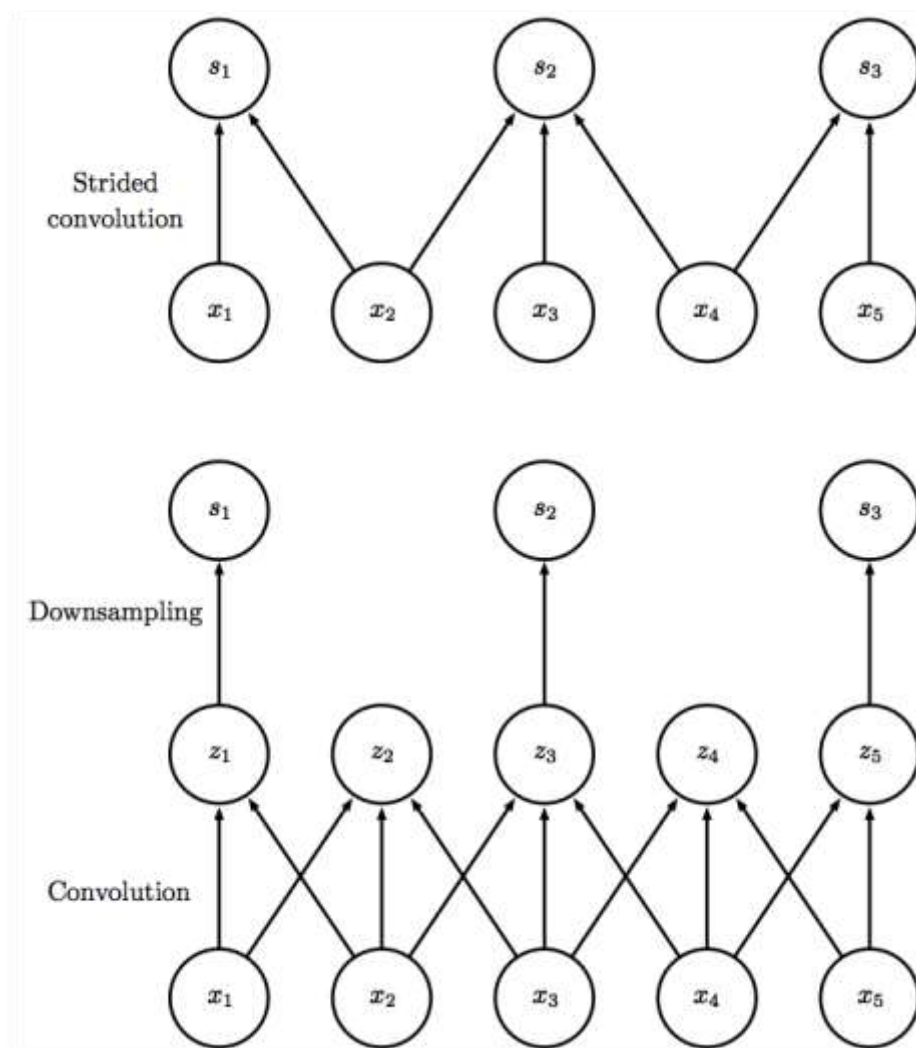
Convolution shares the same parameters across all spatial locations

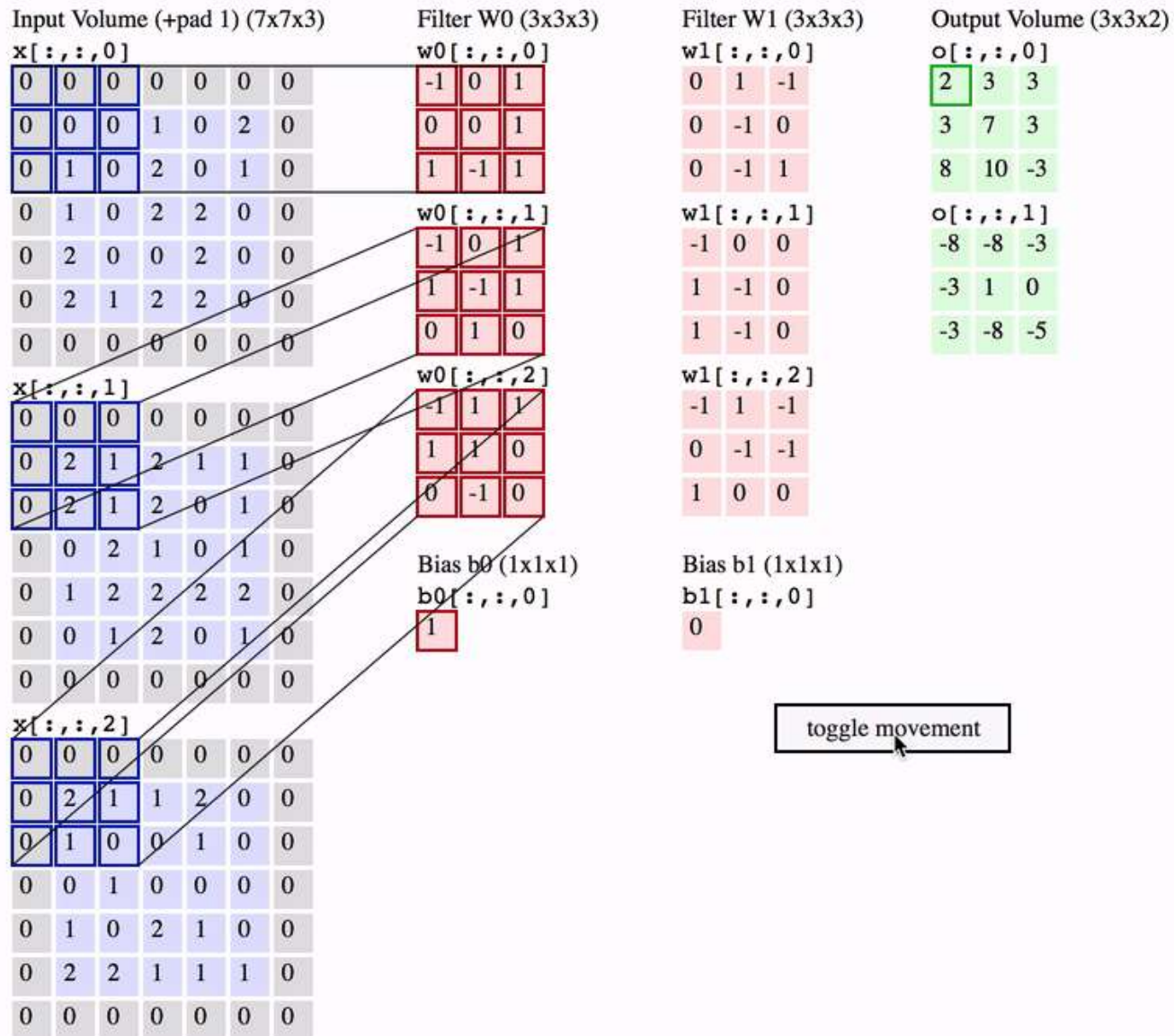


Traditional matrix multiplication does not share any parameters



Convolution with stride

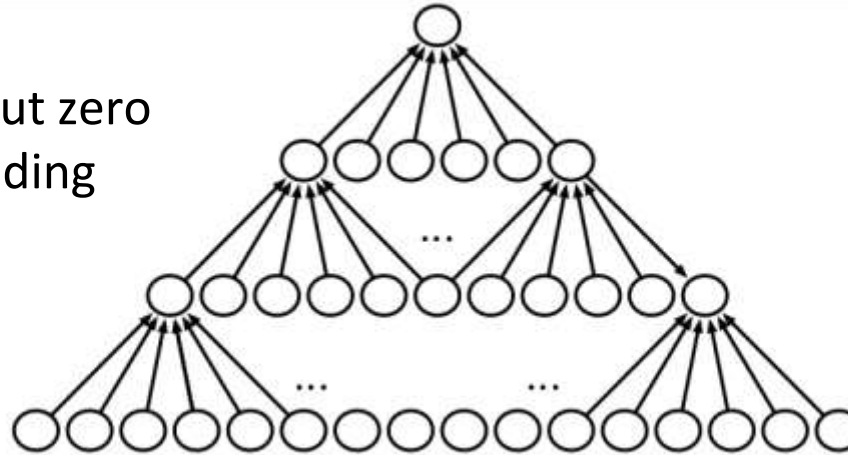




Source: <http://cs231n.github.io/convolutional-networks/>

Zero padding controls size

Without zero padding



With zero padding



Original

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

After 1st conv

4	3	4
2	4	3
2	3	4

After 2nd conv

18

Without padding the image shrinks!

Original

0	0	0	0	0	0	0
0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0

*

0	0	1
0	1	1
1	1	1

=

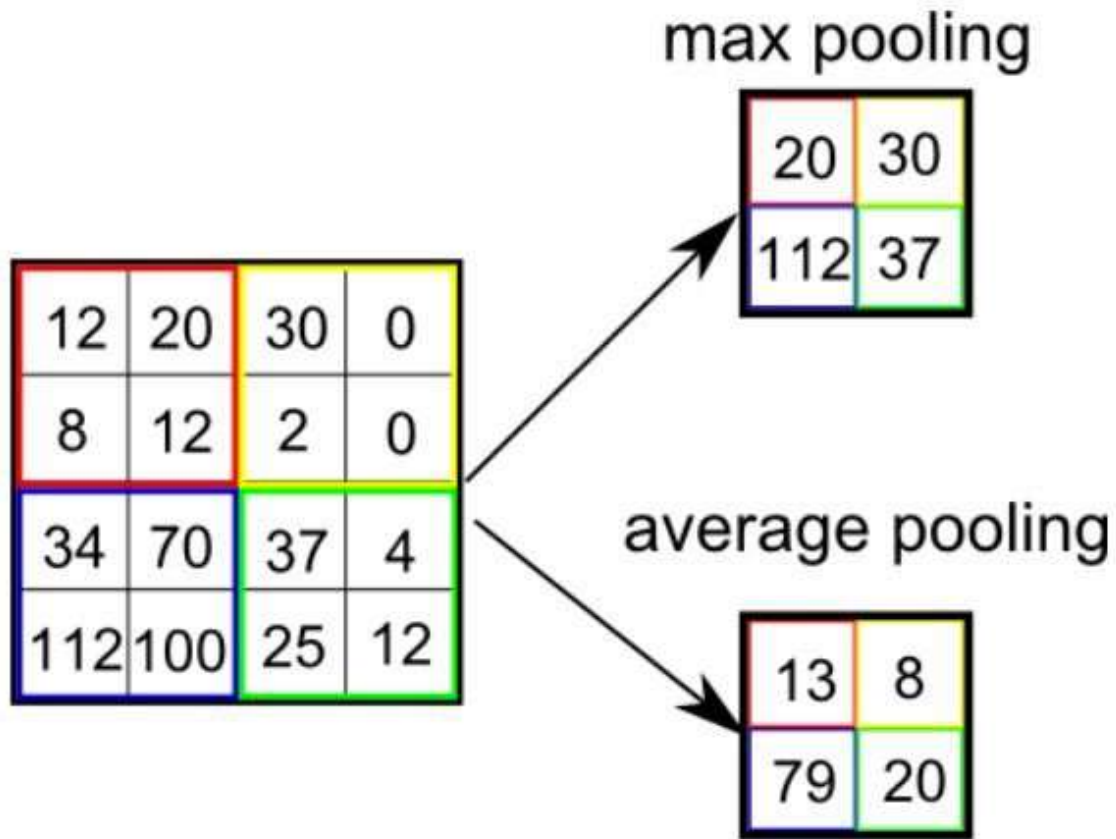
After 1st conv

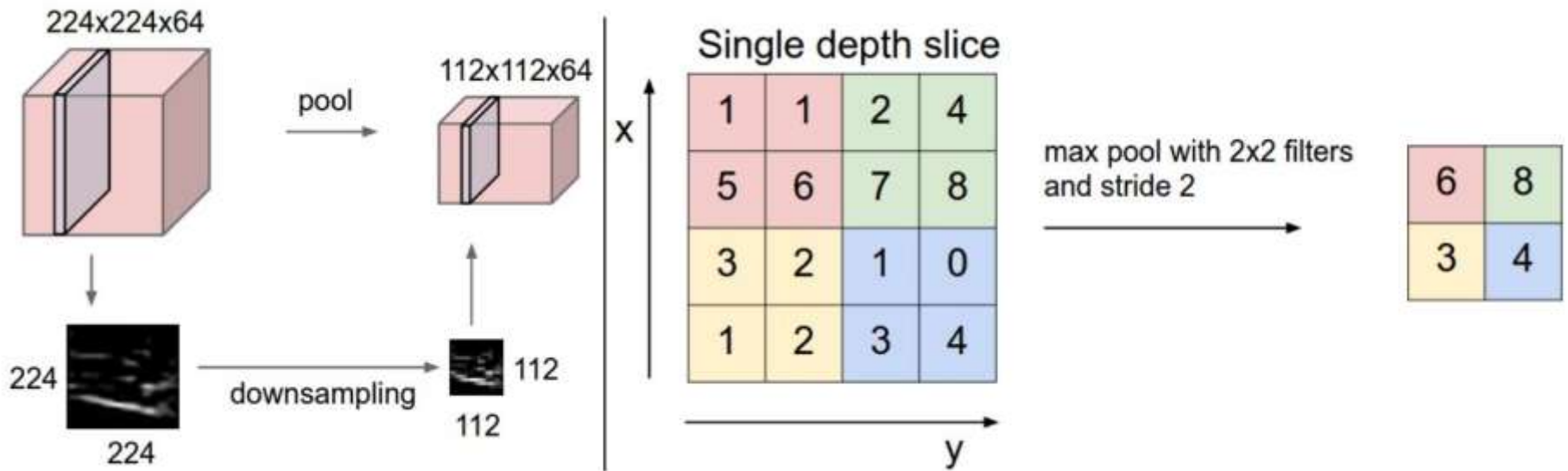
1	1	2	0	0
0	1	1	1	0
0	0	1	2	1
1	0	2	1	0
0	1	1	3	0

With padding the image doesn't shrink!

Pooling

- Aggregate multiple values into a single value
 - Reduce size of layer (speed up computation)
 - Keep most important information for next layer
 - Invariance to small transformation





Pooling layer downsamples the volume spatially, independently in each depth slice of the input volume. Left: In this example, the input volume of size $[224 \times 224 \times 64]$ is pooled with filter size 2, stride 2 into output volume of size $[112 \times 112 \times 64]$. Notice that the volume depth is preserved. Right: The most common downsampling operation is max, giving rise to max pooling, here shown with a stride of 2. That is, each max is taken over 4 numbers (little 2×2 square).

Do we really need pooling?

- Increasing the stride size of the kernel also reduces the size of the image...so why not doing that instead of pooling? Yes you can

Table 2: Model description of the three networks derived from base model C used for evaluating the importance of pooling in case of classification on CIFAR-10 and CIFAR-100. The derived models for base models A and B are built analogously. The higher layers are the same as in Table 1 .

Model		
Strided-CNN-C	ConvPool-CNN-C	All-CNN-C
	Input 32×32 RGB image	
3×3 conv. 96 ReLU	3×3 conv. 96 ReLU	3×3 conv. 96 ReLU
3×3 conv. 96 ReLU with stride $r = 2$	3×3 conv. 96 ReLU 3×3 conv. 96 ReLU	3×3 conv. 96 ReLU
	3×3 max-pooling stride 2	3×3 conv. 96 ReLU with stride $r = 2$
3×3 conv. 192 ReLU	3×3 conv. 192 ReLU	3×3 conv. 192 ReLU
3×3 conv. 192 ReLU with stride $r = 2$	3×3 conv. 192 ReLU 3×3 conv. 192 ReLU	3×3 conv. 192 ReLU
	3×3 max-pooling stride 2	3×3 conv. 192 ReLU with stride $r = 2$
	$\vdots$	

See: <https://arxiv.org/pdf/1412.6806.pdf> and

<https://towardsdatascience.com/iclr-2015-striving-for-simplicity-the-all-convolutional-net-with-interactive-code-manual-b4976e206760>

Summary

- The concept of convolutional NN
 - Why it is helpful in image recognition?
- What is convolution
- Convolutional layer and its basic components
- The special characteristics of convolutional layer
 - Sparse connectivity
 - Parameter sharing
 - Pooling: different type of Invariance

Next Lecture:

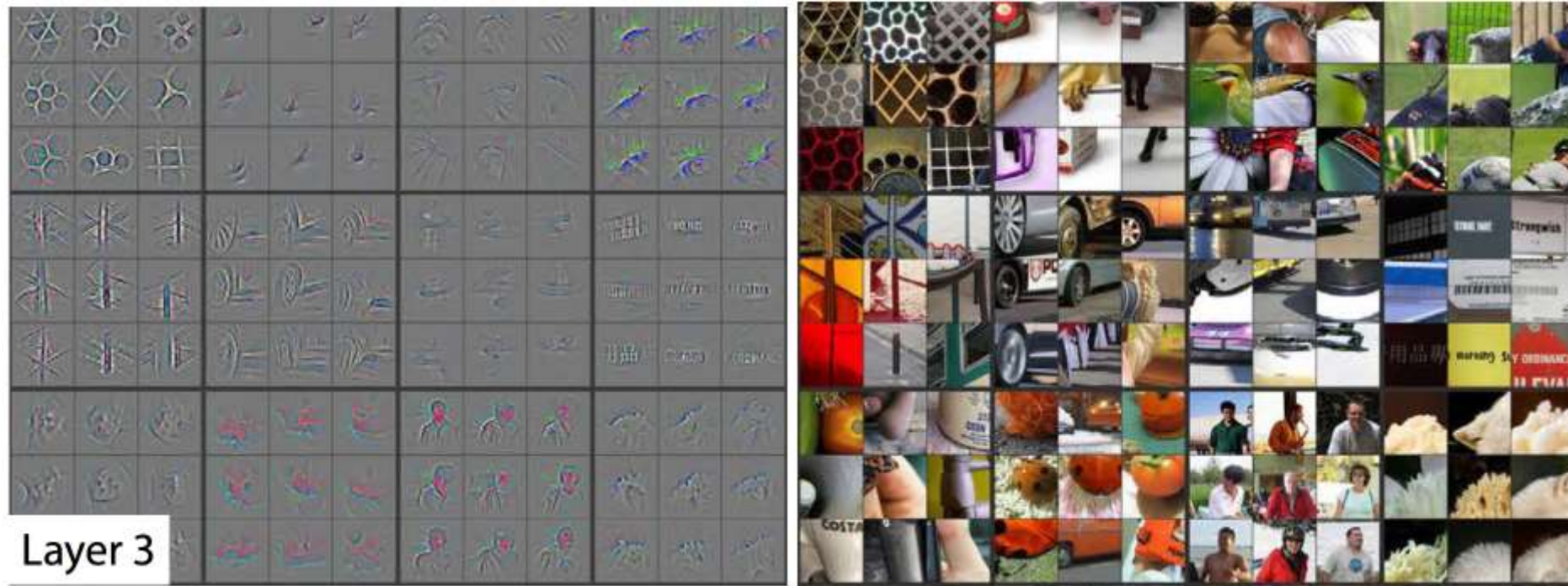
- Variants of CNN and the introduction of RNN

Advanced Deep Learning

2. Convolutional Neural Networks* (Continued)

Prof. Jianguo Zhang
SUSTech

Visualising ConvNets



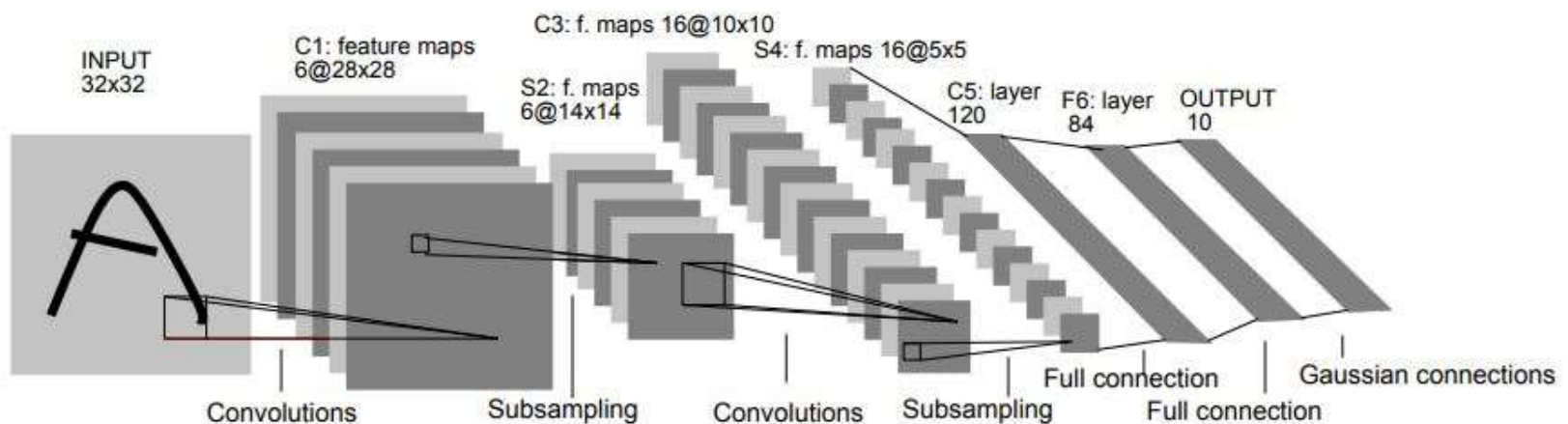
Given a feature map (i.e., convolved image), what are the top 9 activations?

For the full set of experiments see <https://cs.nyu.edu/~fergus/papers/zeilerECCV2014.pdf>

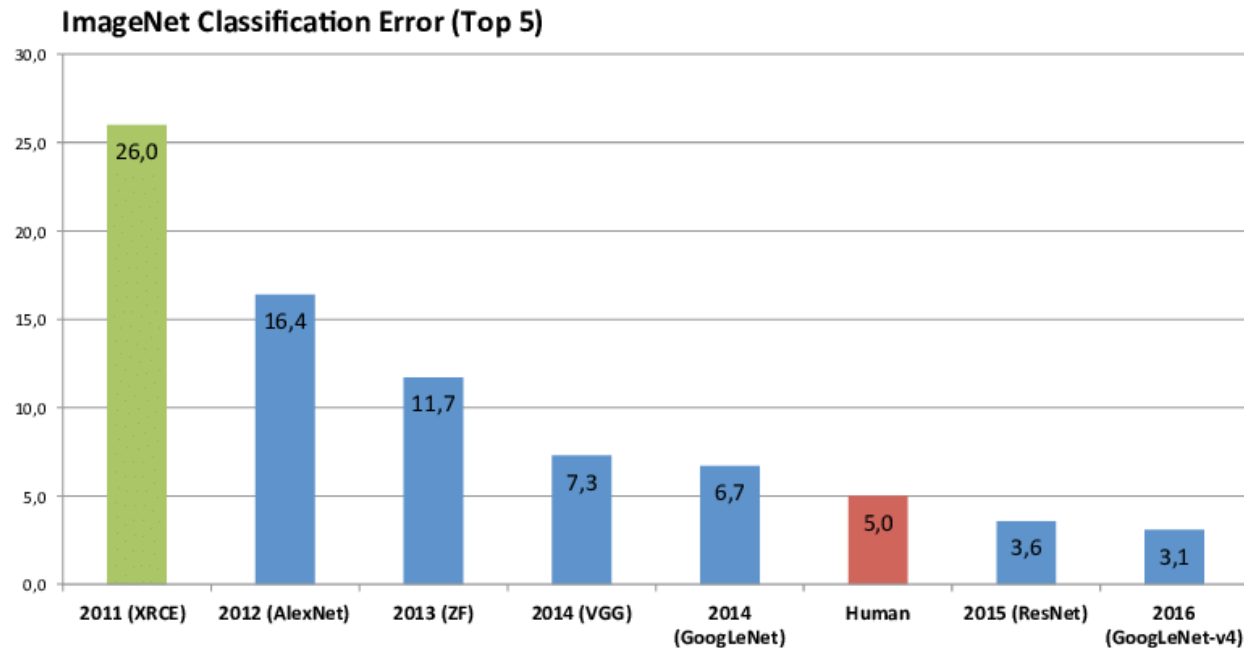
More cool visualisations here: <https://towardsdatascience.com/applied-deep-learning-part-4-convolutional-neural-networks-584bc134c1e2>

The granddaddy of CNNs: LeNet-5

- 7-level convolutional network by LeCun et al in 1998 (<http://yann.lecun.com/exdb/lenet/>)
- Applied to recognise hand-written numbers on cheques (32 x 32), but due to computational constraints, couldn't scale up to larger images

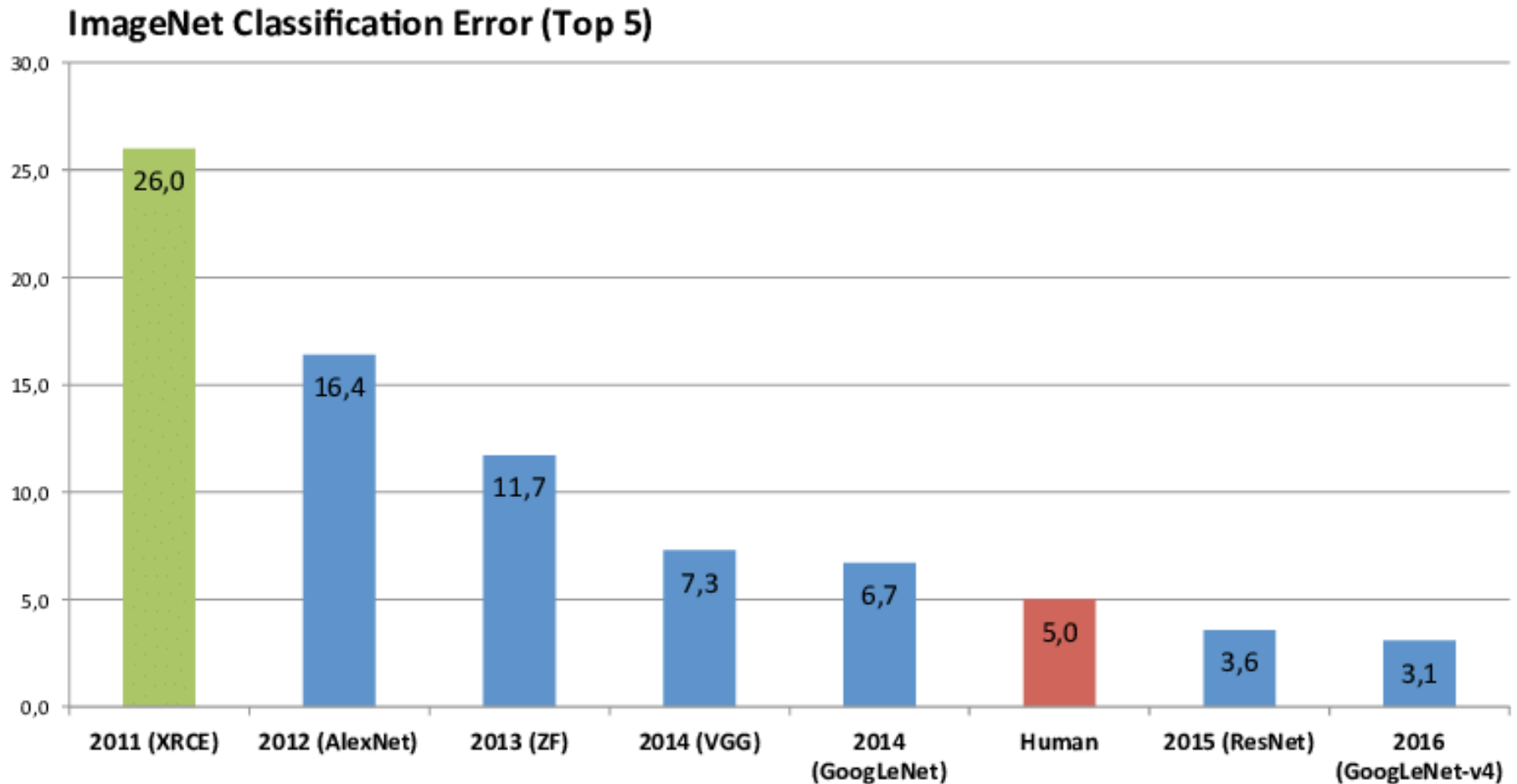


The ImageNet challenge

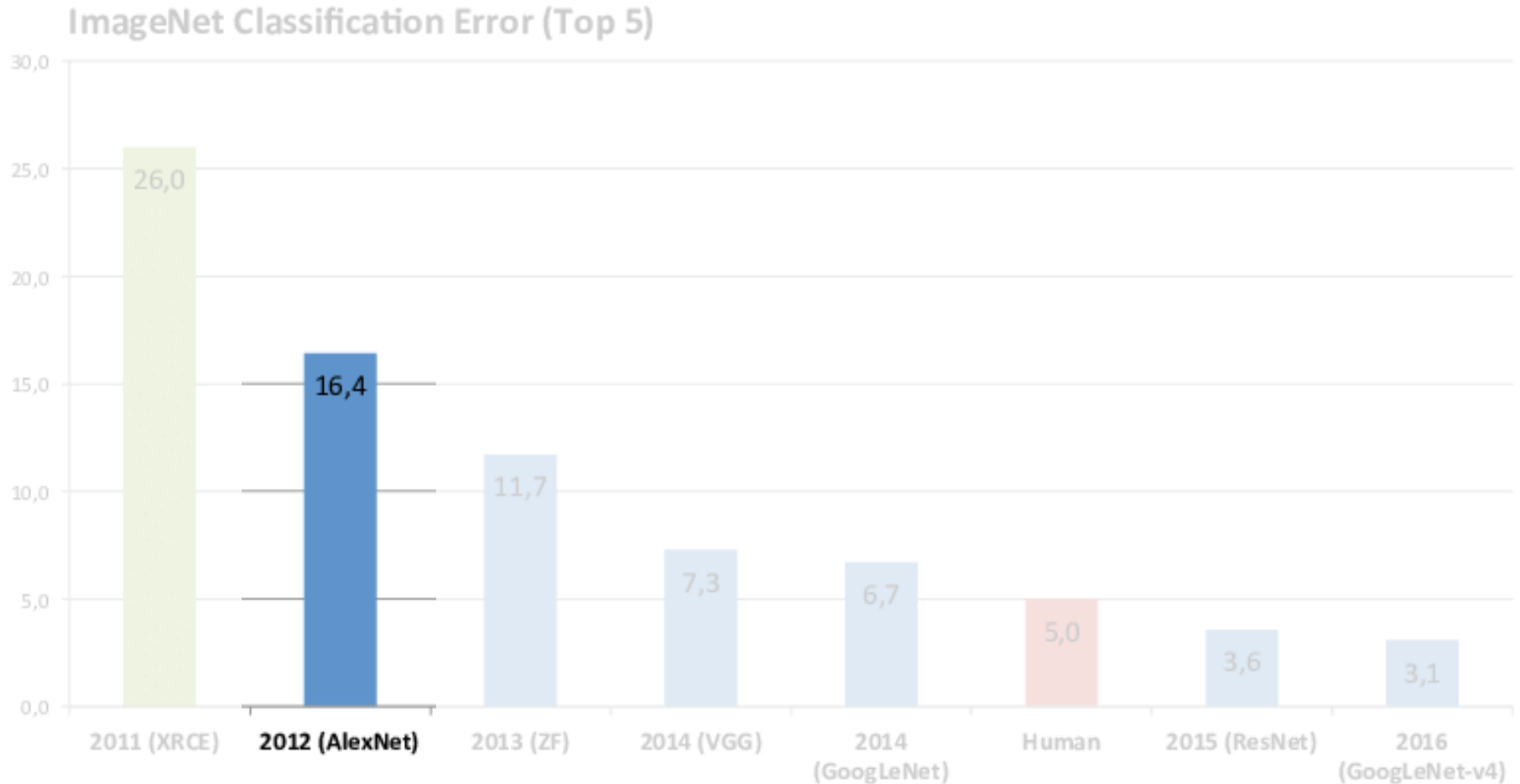


The ImageNet project is a large visual database designed for use in visual object recognition software research. The ImageNet project runs an annual software contest, the **ImageNet Large Scale Visual Recognition Challenge** (ILSVRC)

The ImageNet challenge



The ImageNet challenge



AlexNet

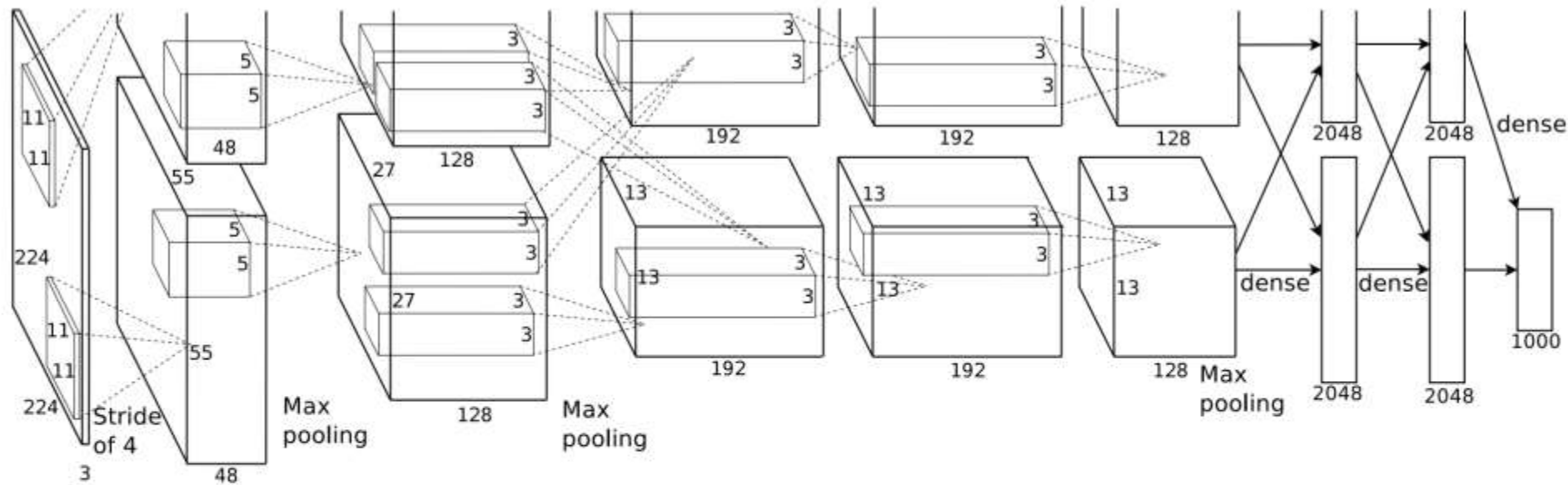
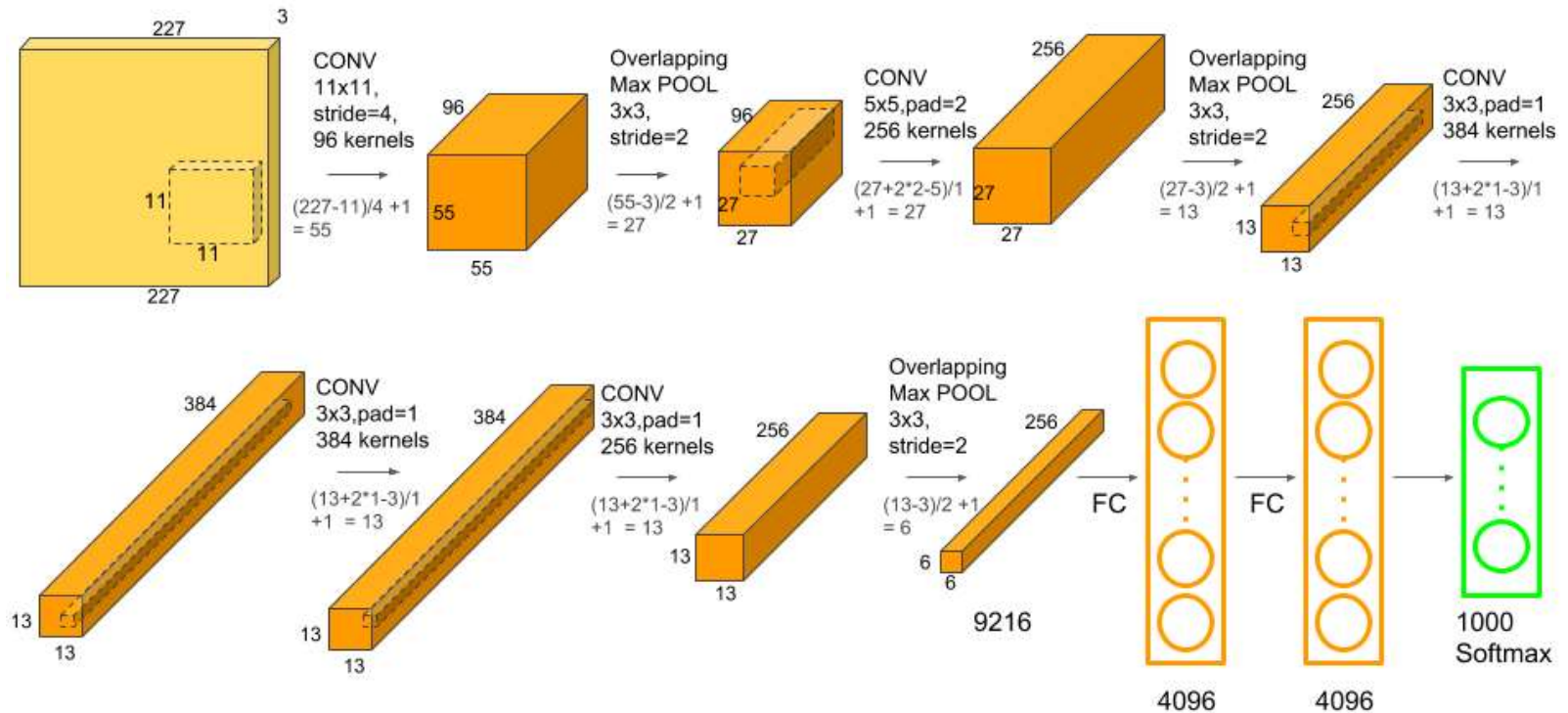


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

AlexNet

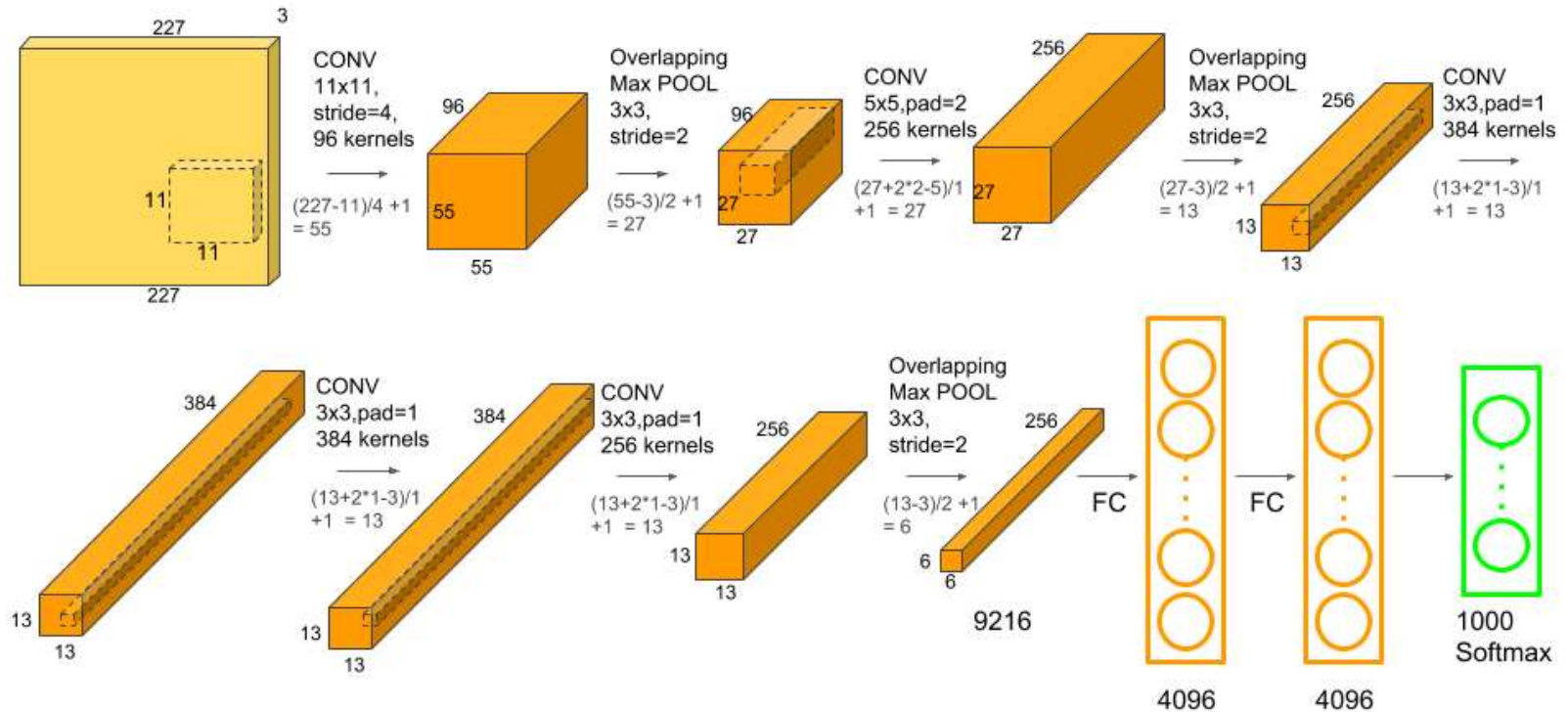
- Outperformed all competitors and won the challenge by reducing **top-5 error** from 26% to **16.4%**
- **2nd place** went to a non-CNN model with **top-5 error** rate around **26.2%**
- Approximately 60 million parameters
- Paper: <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf>

AlexNet



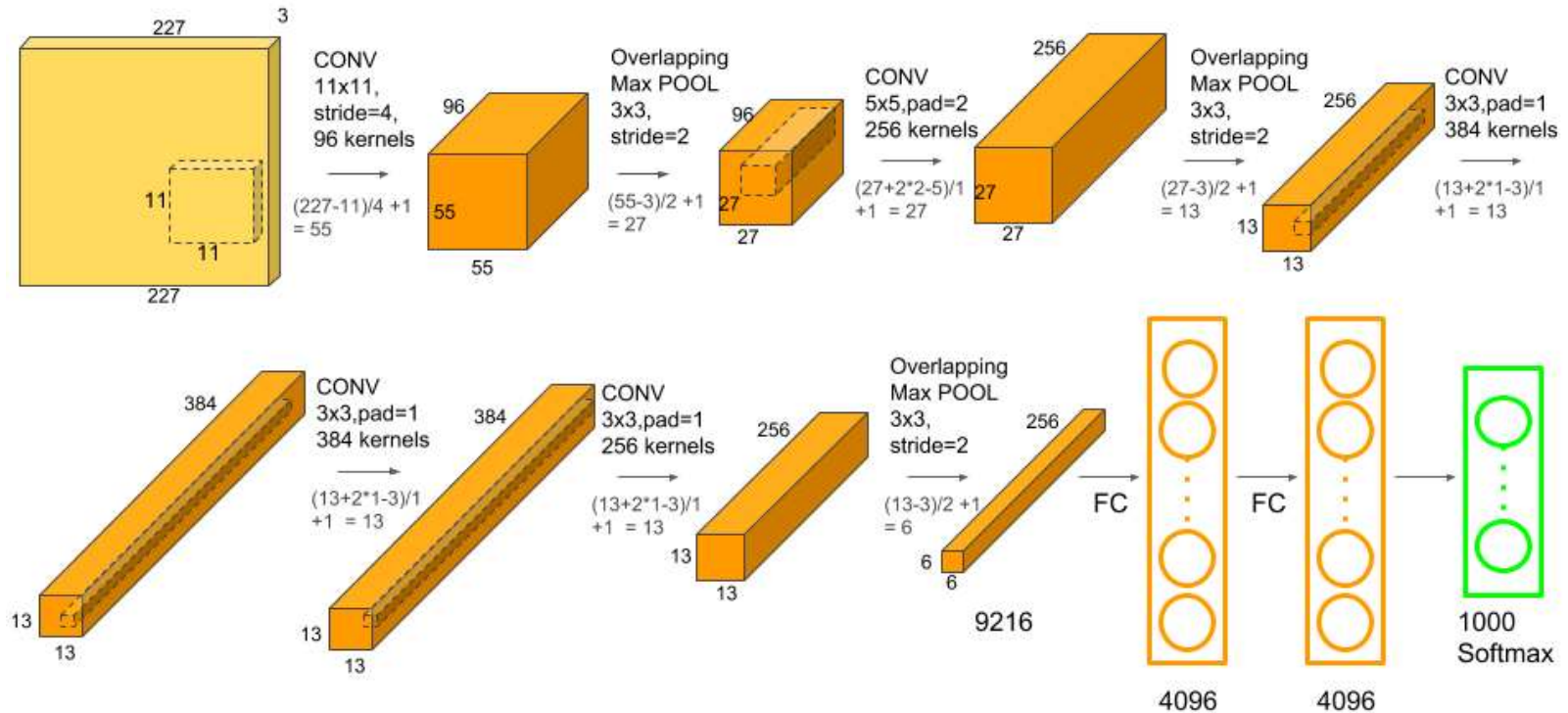
ReLU instead of tanh (LeNet) + dropout

AlexNet



5 convolutional layers + 3 fully connected layers

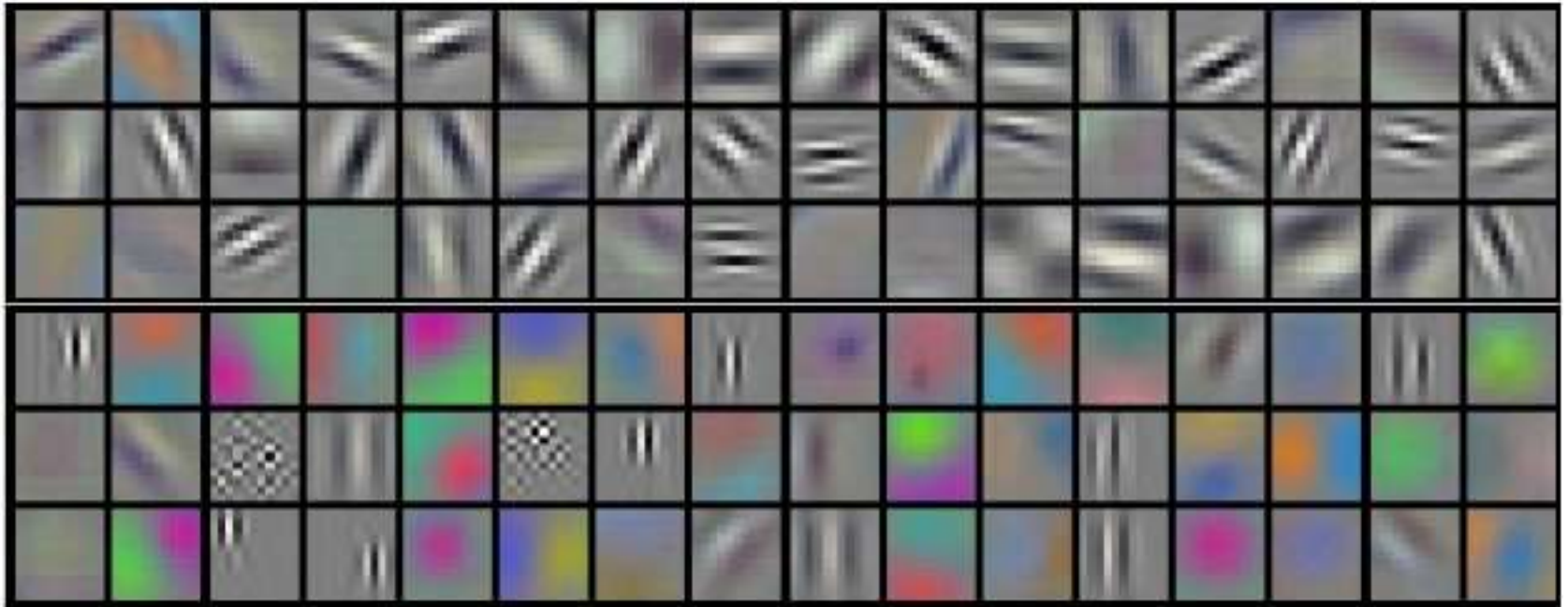
AlexNet



Trained for 6 days simultaneously on two Nvidia Geforce GTX 580 GPUs:
 kernels of the 2nd, 4th, and 5th conv layers are connected only to kernel maps in the previous layer residing on the same GPU. Kernels of the 3rd conv layer are connected to all kernel maps in the second layer.

source: <https://www.learnopencv.com/understanding-alexnet/>

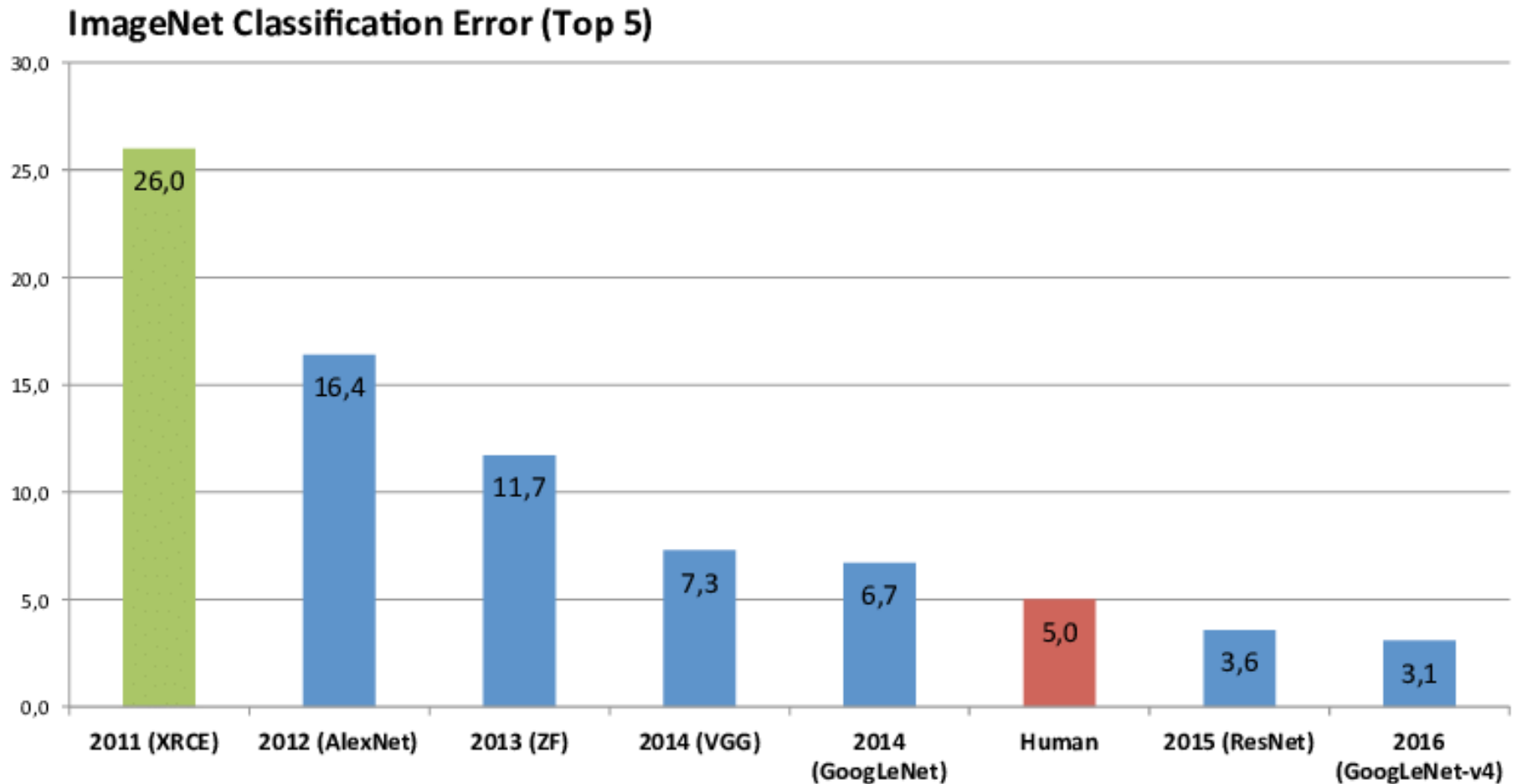
AlexNet



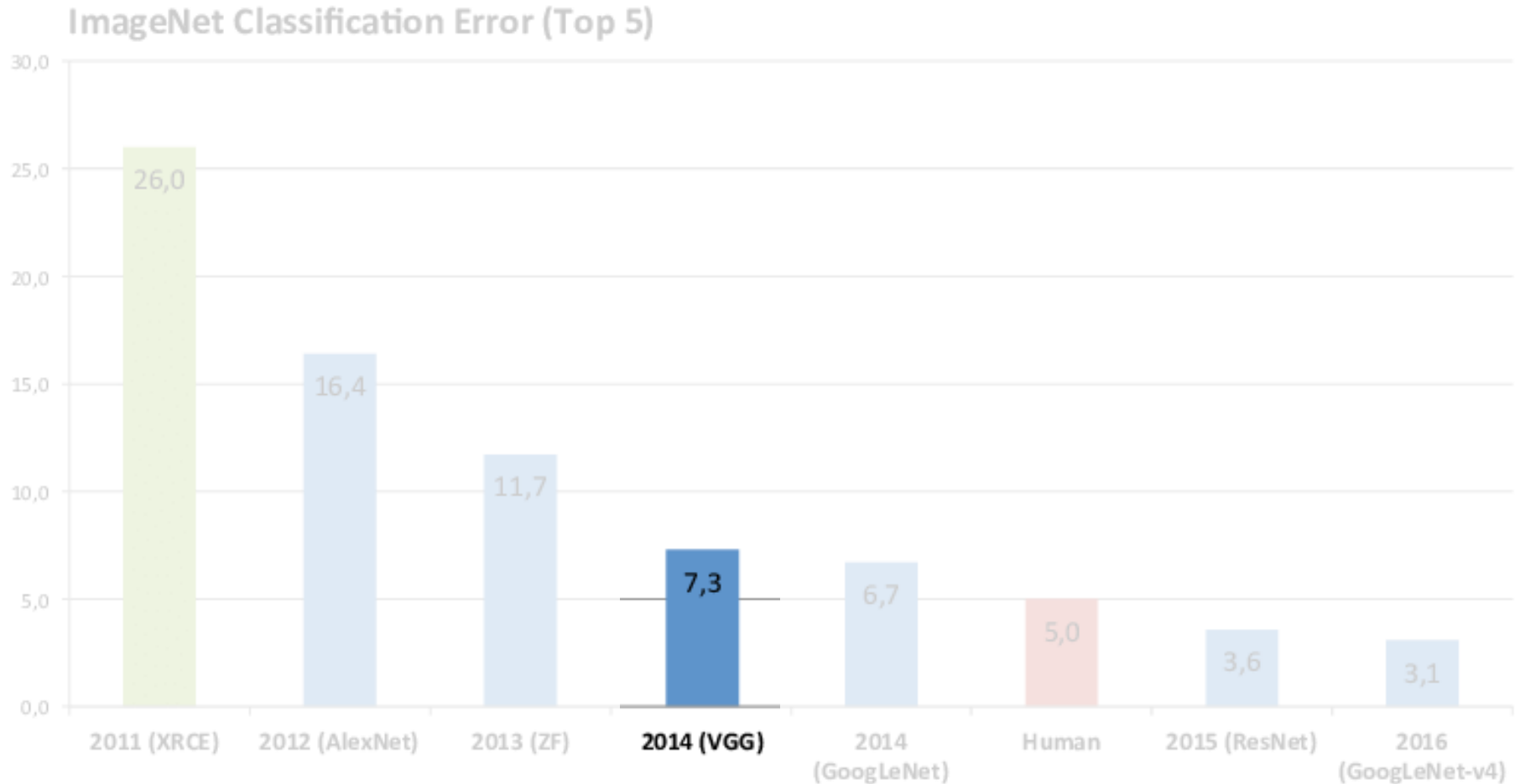
96 convolutional kernels of size $11 \times 11 \times 3$ learned by the first convolutional layer on the $224 \times 224 \times 3$ input images. The top 48 kernels were learned on GPU 1 while the bottom 48 kernels were learned on GPU 2.

The effect of splitting the learning of the kernels over two GPUs can be observed in the difference between the top and bottom sets of kernel filters above: remember, these are learned and not hardcoded!

The ImageNet challenge



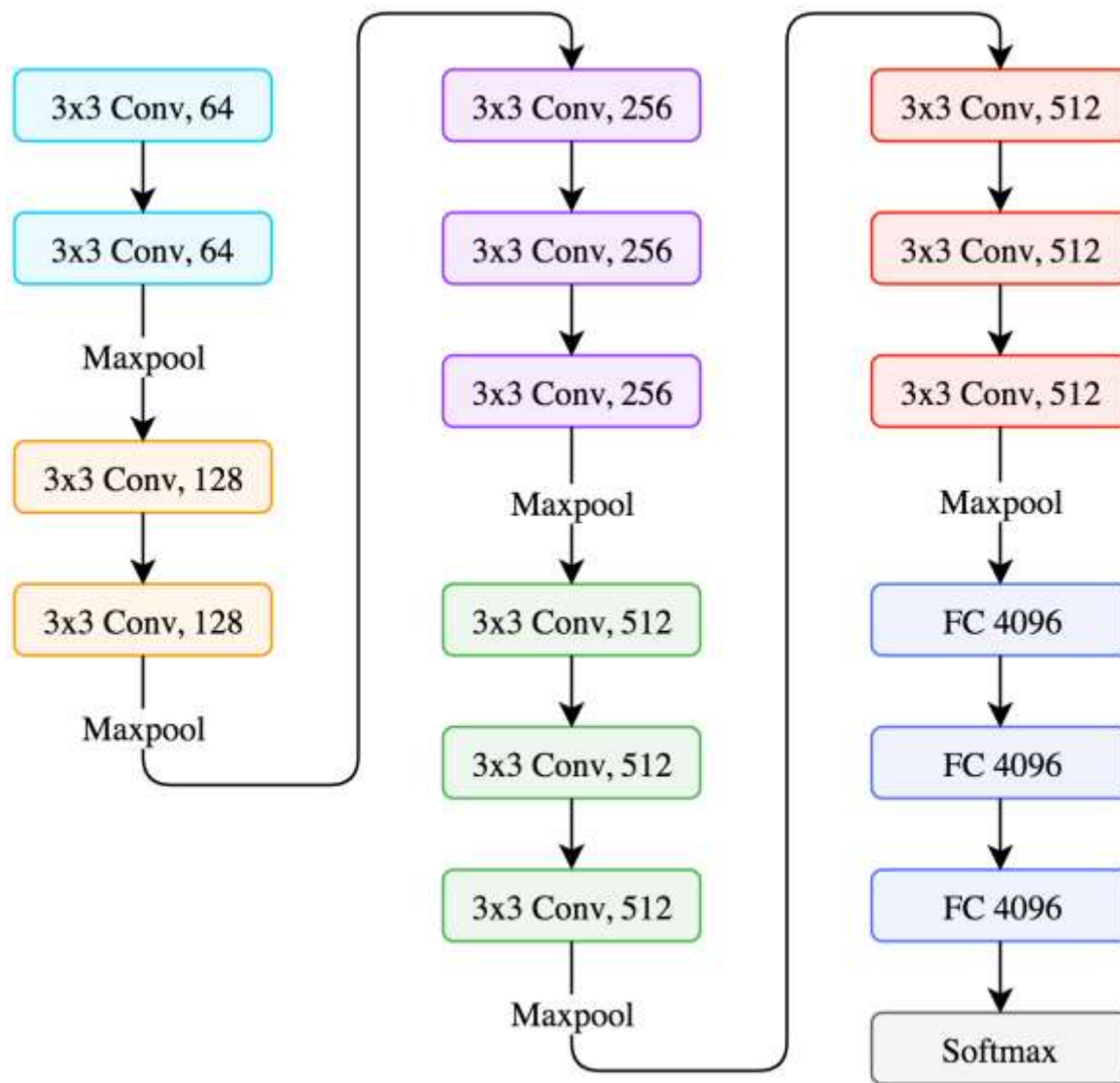
The ImageNet challenge





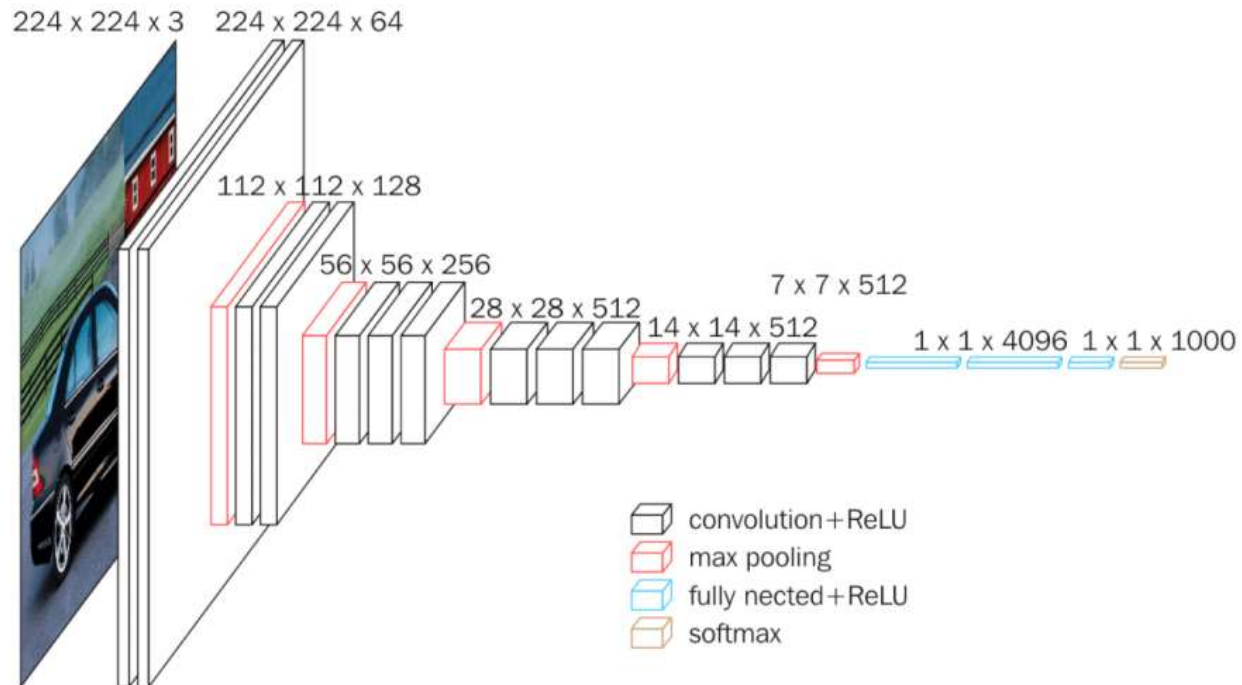
VGG16

- Runner-up at the 2014 ILSVRC (won by GoogLeNet) with a 7.3% error rate, developed by Oxford's **V**isual **G**eometry **G**roup
- **16 convolutional layers** (lots of 3x3 filters)
- Approximately 138 million parameters
- Trained on 4 GPUs for 3 weeks
- “Simple” yet effective (depth is the key)
- Paper: <https://arxiv.org/abs/1409.1556>



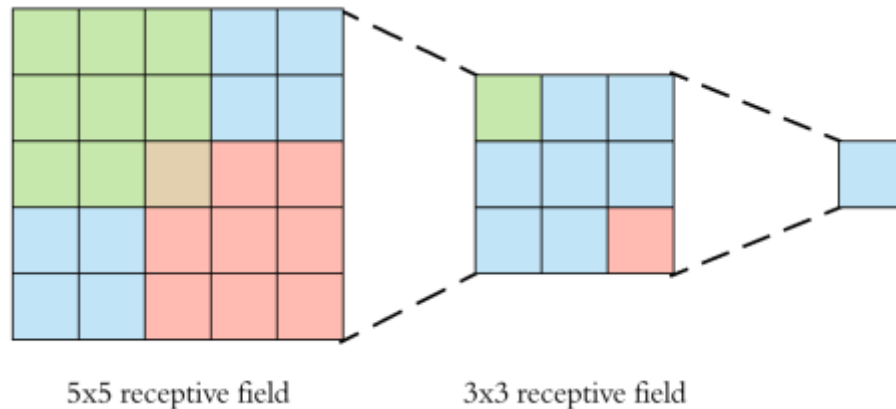
Source: <https://towardsdatascience.com/applied-deep-learning-part-4-convolutional-neural-networks-584bc134c1e2>

- VGG network structure



VGG16: key observations

- Number of filters increases as we go deeper
- 3 x 3 convolutions stacked in groups of 2 or 3
- Note: 2 stacked 3 x 3 convolutions have the receptive field of a single 5 x 5 convolution

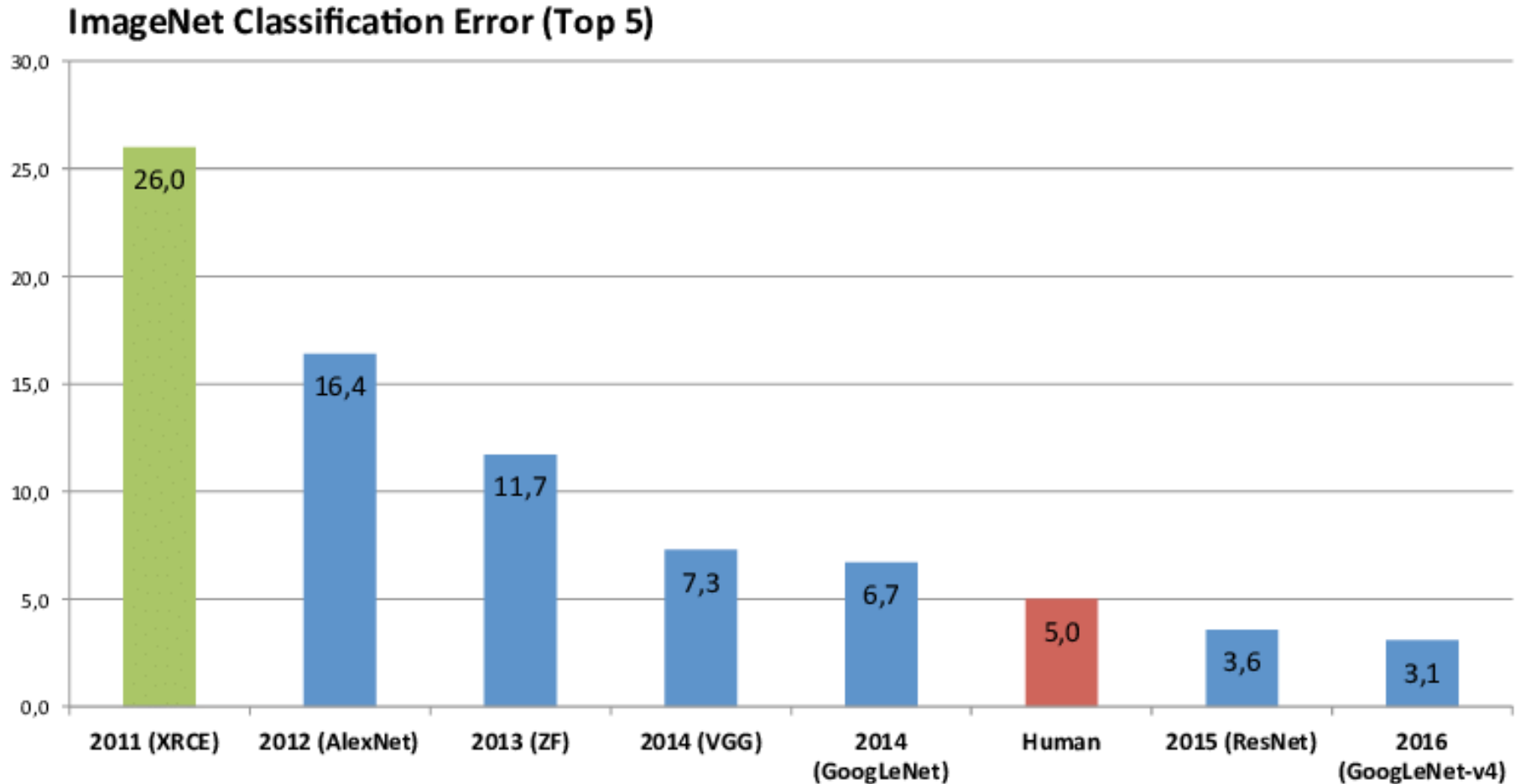


What about 3 stacked 3 x 3 convolutions?

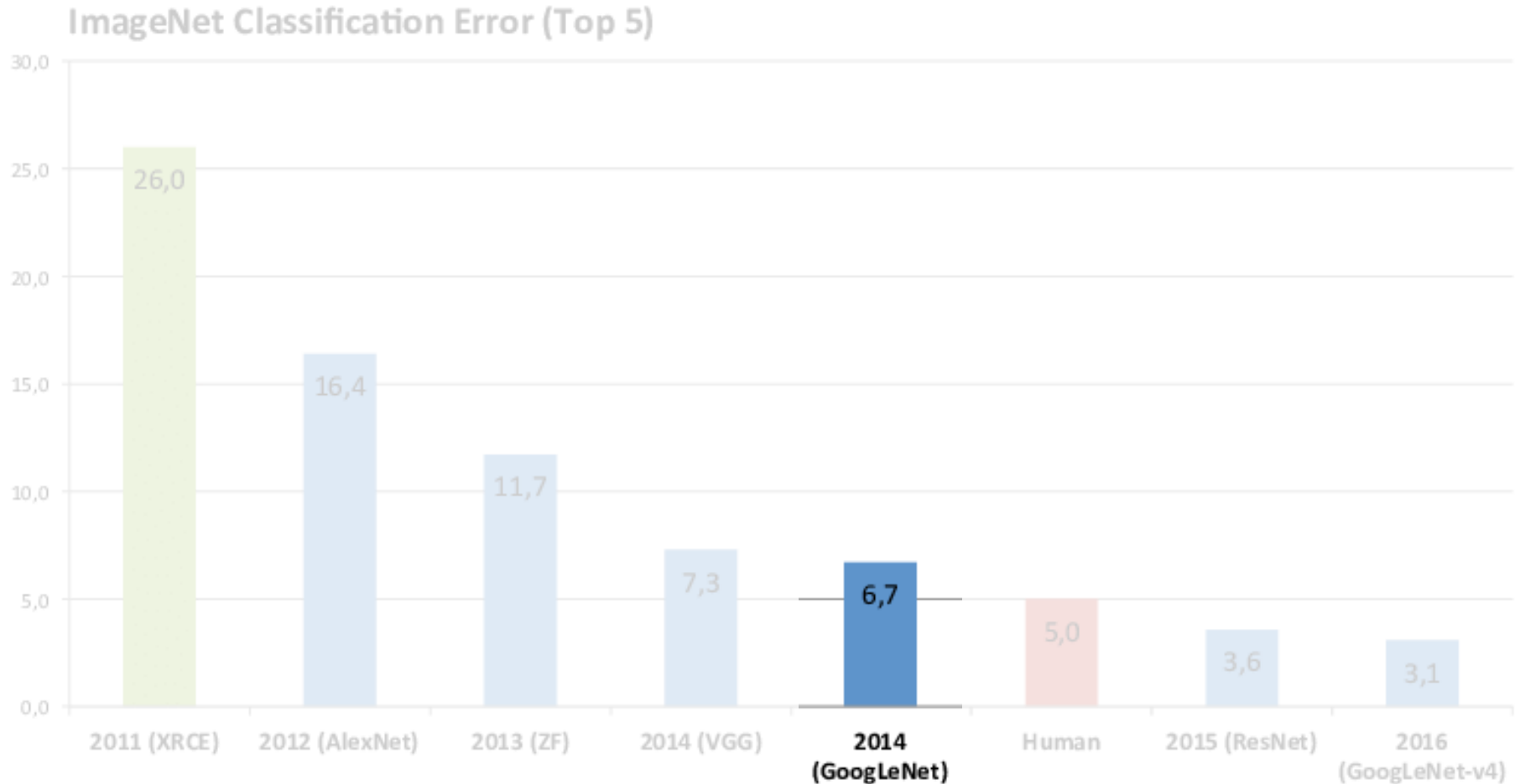
VGG16: key observations

- Number of filters increases as we go deeper
- 3 x 3 convolutions stacked in groups of 2 or 3
- Note: 2 stacked 3 x 3 convolutions have the receptive field of a single 5 x 5 convolution
- **Why stacking several small filters??**
 1. More ReLU operations generate more non-linearity which means a more powerful model
 2. Fewer parameters, e.g.,
 $(3 \times 3) \times 3 = 27$ versus $(7 \times 7) = 49$

The ImageNet challenge



The ImageNet challenge



GoogLeNet (Inception network V1)

- The winner of the 2014 ILSVRC was actually GoogLeNet (developed by Google), built around the **Inception** unit, with top-5 error rate of 6.7%
- Very close to human-level performance (5%)
- Key features: inception units, batch normalization, image distortions, RMSprop
- **Parameters: ~5 million (V1) and ~23 million (V3)**
- Papers: <https://arxiv.org/abs/1409.4842> and <https://arxiv.org/abs/1512.00567>

GoogLeNet (Inception network V1)

- Let's first try to understand the problem the engineers behind the Inception unit tried to solve:
 1. Very deep networks are prone to overfitting due to the large number of parameters
 2. Naively stacking large convolution operations is computationally expensive: if two convolutional layers are chained, any uniform increase in the number of their filters results in a quadratic increase of computation

3) Salient parts in images can have very large variation in size



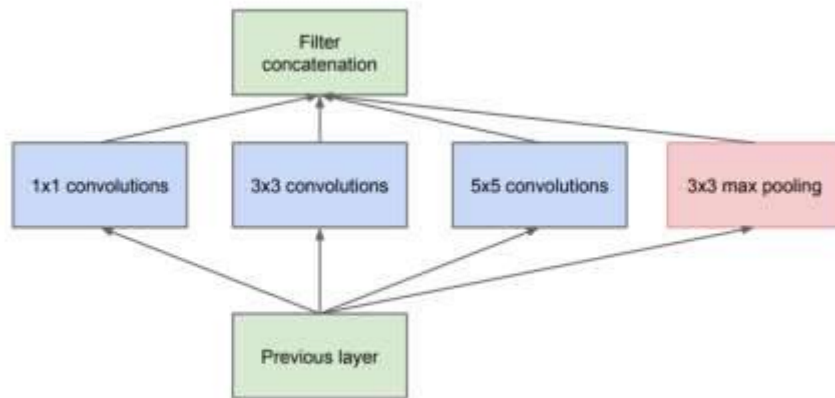
How to choose the kernel size then?



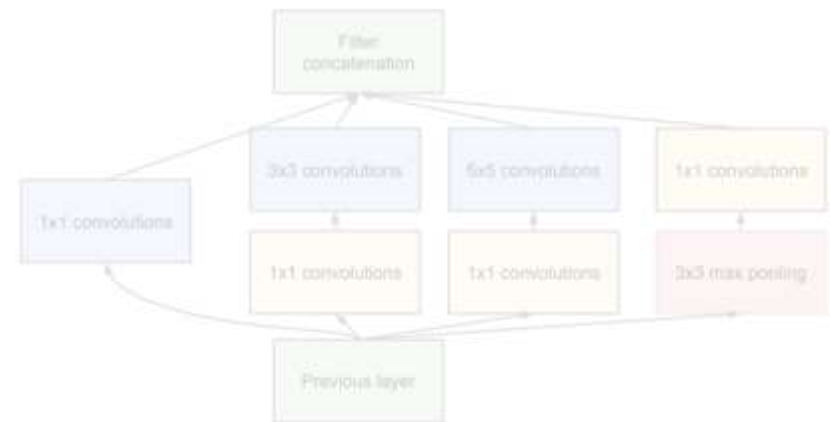
Source: <https://towardsdatascience.com/a-simple-guide-to-the-versions-of-the-inception-network-7fc52b863202>

GoogLeNet (Inception network V1)

- Solution: make network **wider instead of deeper** having filters of **multiple sizes** operating at the **same level**



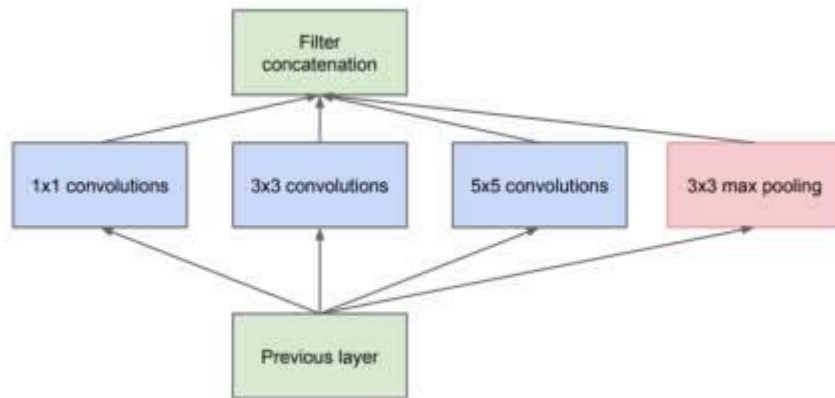
(a) Inception module, naïve version



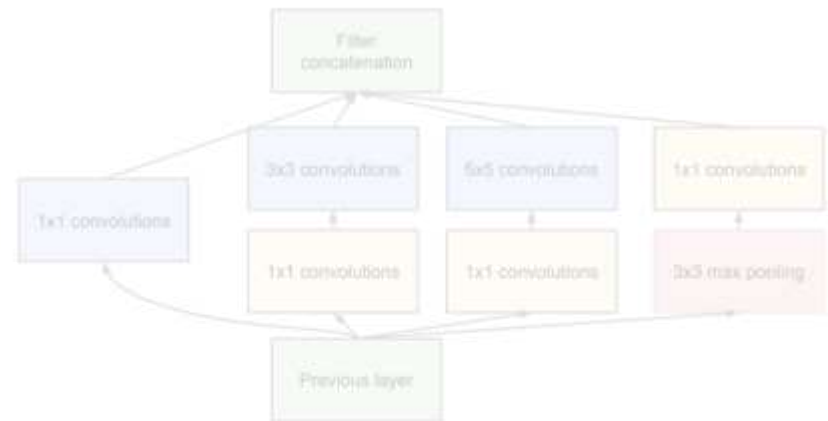
(b) Inception module with dimension reductions

GoogLeNet (Inception network V1)

- Solution: make network **wider instead of deeper** having filters of **multiple sizes** operating at the **same level**



(a) Inception module, naïve version

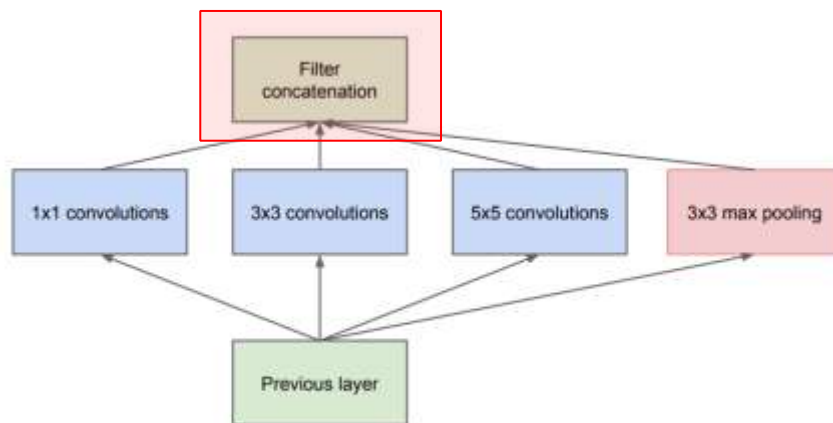


(b) Inception module with dimension reductions

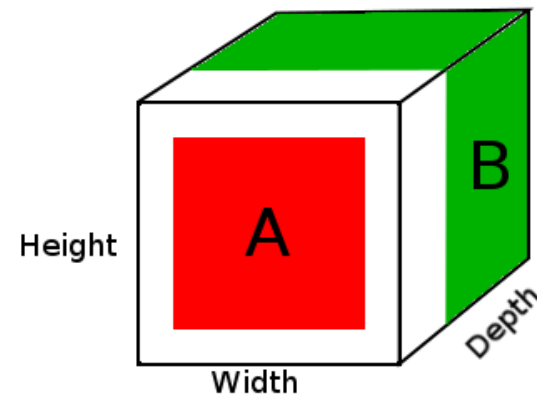
Use padding to make sure filter outputs have same size. Concatenate along depth.

GoogLeNet (Inception network V1)

- Solution: make network **wider instead of deeper** having filters of **multiple sizes** operating at the **same level**



(a) Inception module, naïve version

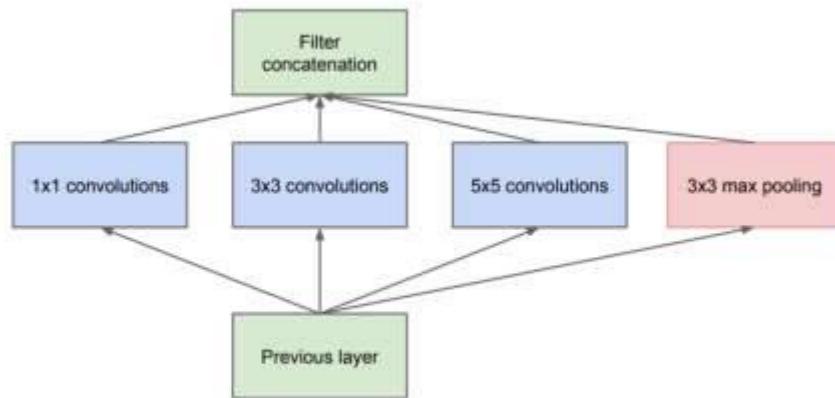


```
A = tensor of size (14, 14, 2)
B = tensor of size (16, 16, 3)
result = DepthConcat([A, B])
```

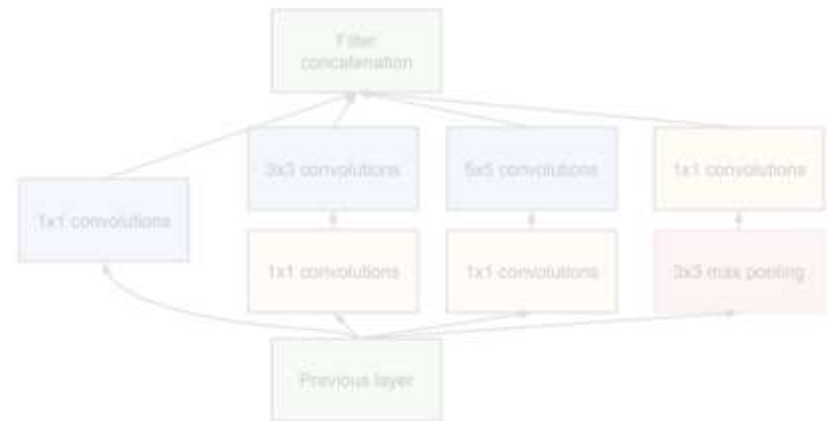
Use padding to make sure filter outputs have same size. Concatenate along depth.

GoogLeNet (Inception network V1)

- Solution: make network **wider instead of deeper** having filters of **multiple sizes** operating at the **same level**



(a) Inception module, naïve version

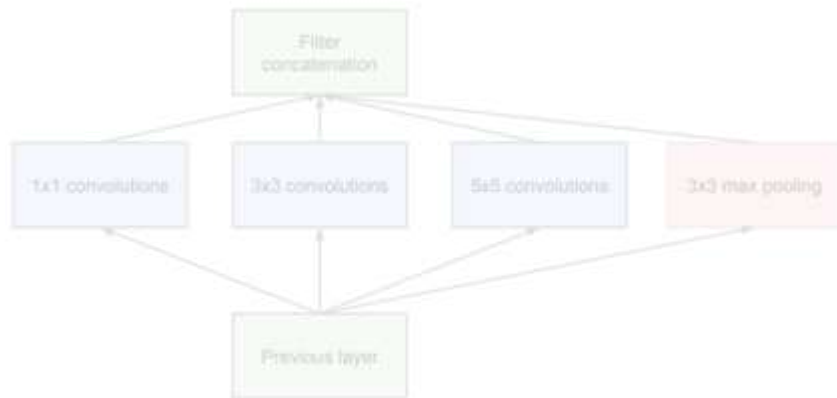


(b) Inception module with dimension reductions

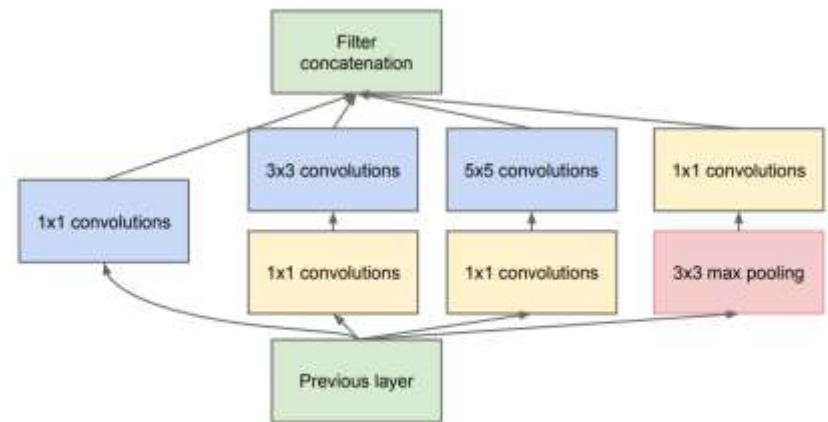
This naive version of the Inception unit is very expensive!

GoogLeNet (Inception network V1)

- Solution: make network **wider instead of deeper** having filters of **multiple sizes** operating at the **same level**



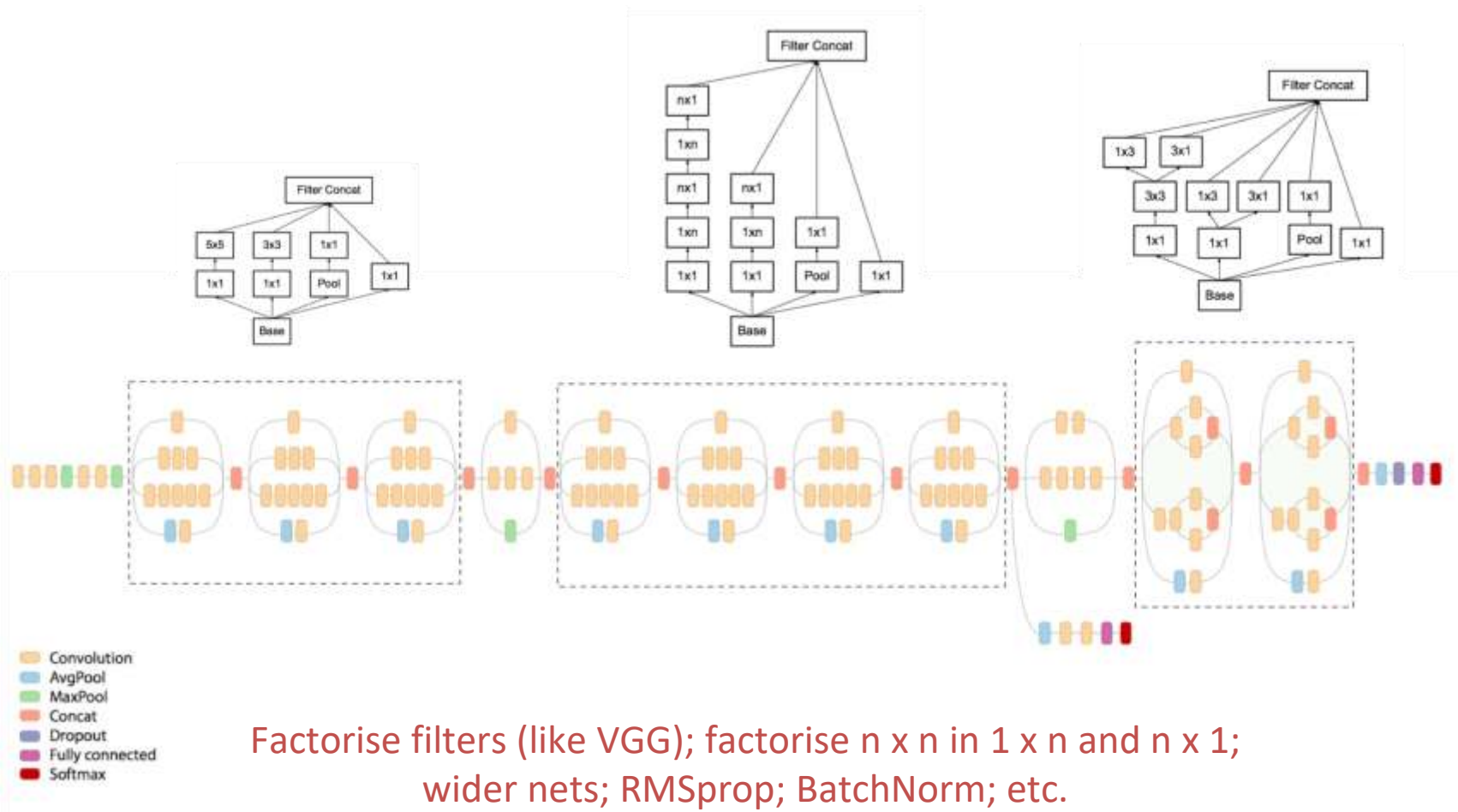
(a) Inception module, naïve version



(b) Inception module with dimension reductions

Solution: use 1x1 convolutions to limit the number of channels

Inception network V3



Factorise filters (like VGG); factorise $n \times n$ in $1 \times n$ and $n \times 1$;
wider nets; RMSprop; BatchNorm; etc.
(results in 23 million parameters)

ResNet: ILSVRC 2015 winner

Revolution of Depth

AlexNet, 8 layers
(ILSVRC 2012)



VGG, 19 layers
(ILSVRC 2014)



ResNet, 152 layers
(ILSVRC 2015)

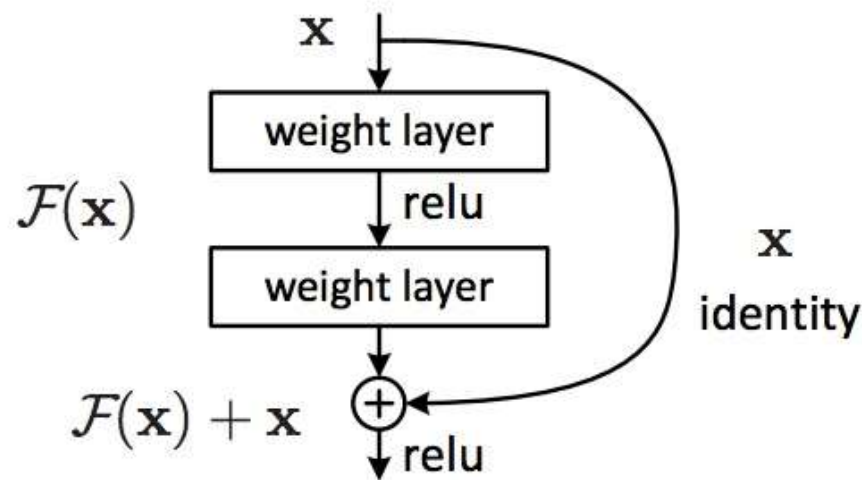


ICCV15

K. He, X. Zhang, S. Ren, and J. Sun, [Deep Residual Learning for Image Recognition](#),
CVPR 2016 (Best Paper)

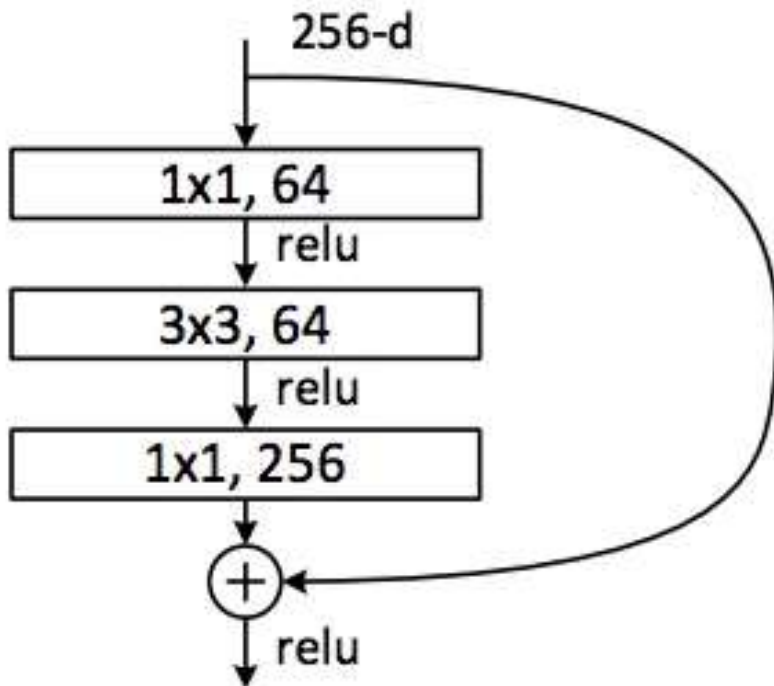
ResNet

- The residual module
 - Introduce *skip* or *shortcut* connections (existing before in various forms in literature)
 - Make it easy for network layers to represent the identity mapping



ResNet

Deeper residual module (bottleneck)



- Directly performing 3x3 convolutions with 256 feature maps at input and output:
 $256 \times 256 \times 3 \times 3 \sim 600K$ operations
- Using 1x1 convolutions to reduce 256 to 64 feature maps, followed by 3x3 convolutions, followed by 1x1 convolutions to expand back to 256 maps:
 $256 \times 64 \times 1 \times 1 \sim 16K$
 $64 \times 64 \times 3 \times 3 \sim 36K$
 $64 \times 256 \times 1 \times 1 \sim 16K$
Total: $\sim 70K$

ResNet

- Architectures for ImageNet:

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times 3, 64 \\ 3\times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 64 \\ 3\times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3\times 3, 128 \\ 3\times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 128 \\ 3\times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3\times 3, 256 \\ 3\times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 256 \\ 3\times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3\times 3, 512 \\ 3\times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 512 \\ 3\times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

K. He, X. Zhang, S. Ren, and J. Sun, [Deep Residual Learning for Image Recognition](#),
CVPR 2016 (Best Paper)

Summary: ILSVRC 2012-2015

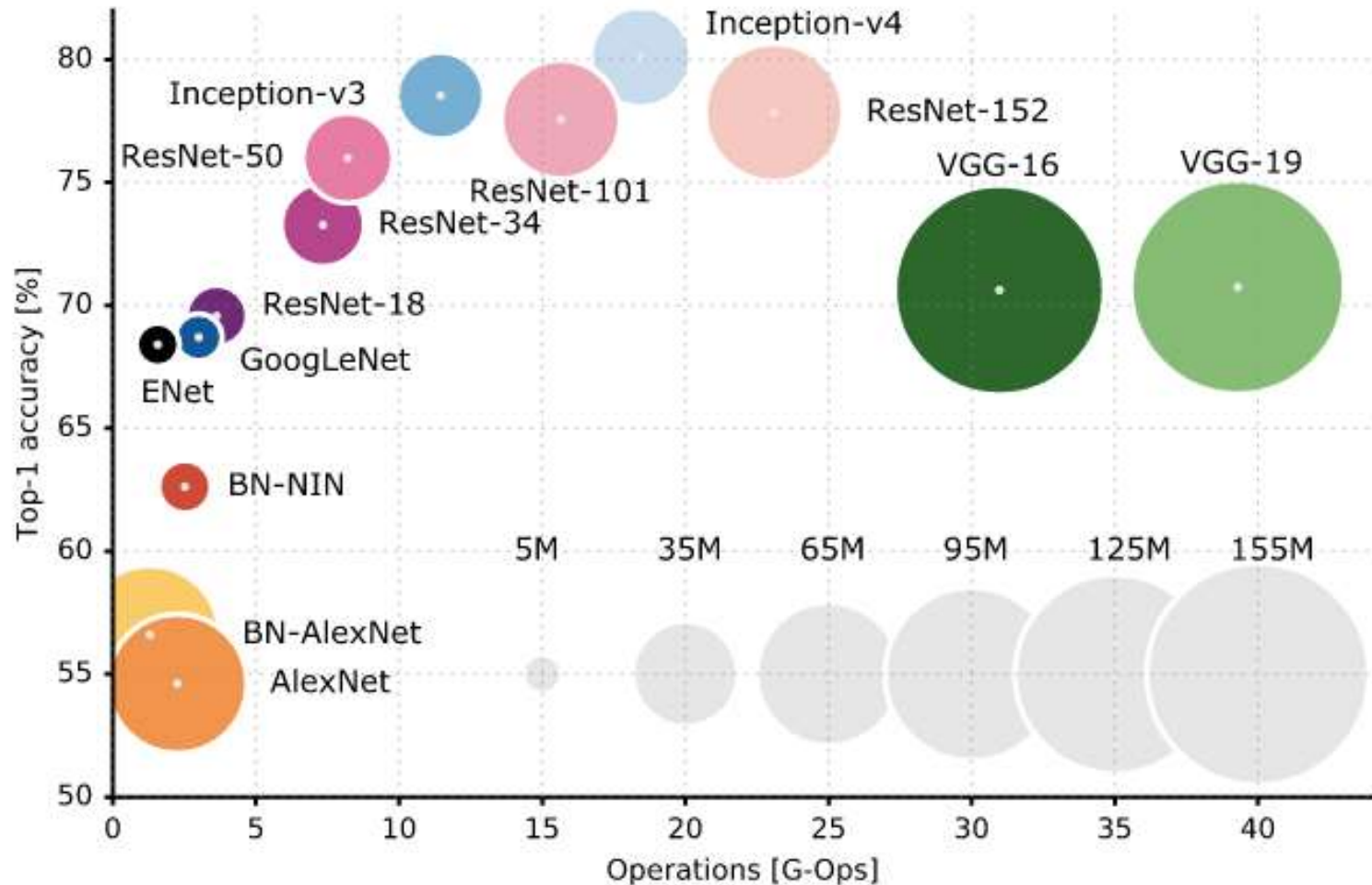
Team	Year	Place	Error (top-5)	External data
SuperVision – Toronto (AlexNet, 7 layers)	2012	-	16.4%	no
SuperVision	2012	1st	15.3%	ImageNet 22k
Clarifai – NYU (7 layers)	2013	-	11.7%	no
Clarifai	2013	1st	11.2%	ImageNet 22k
VGG – Oxford (16 layers)	2014	2nd	7.32%	no
GoogLeNet (19 layers)	2014	1st	6.67%	no
ResNet (152 layers)	2015	1st	3.57%	
Human expert*			5.1%	

<http://karpathy.github.io/2014/09/02/what-i-learned-from-competing-against-a-convnet-on-imagenet/>

What's missing from the picture?

- Training tricks and details: initialization, regularization, normalization
- Training data augmentation
- Averaging classifier outputs over multiple crops/flips
- Ensembles of networks
- Officially, starting with 2015, image classification is not part of ILSVRC challenge, but people continue to benchmark on the data

Comparing architectures



Interpreting ResNets

- ResNets are collections of many paths of different length, and shorter paths predominantly contribute to training

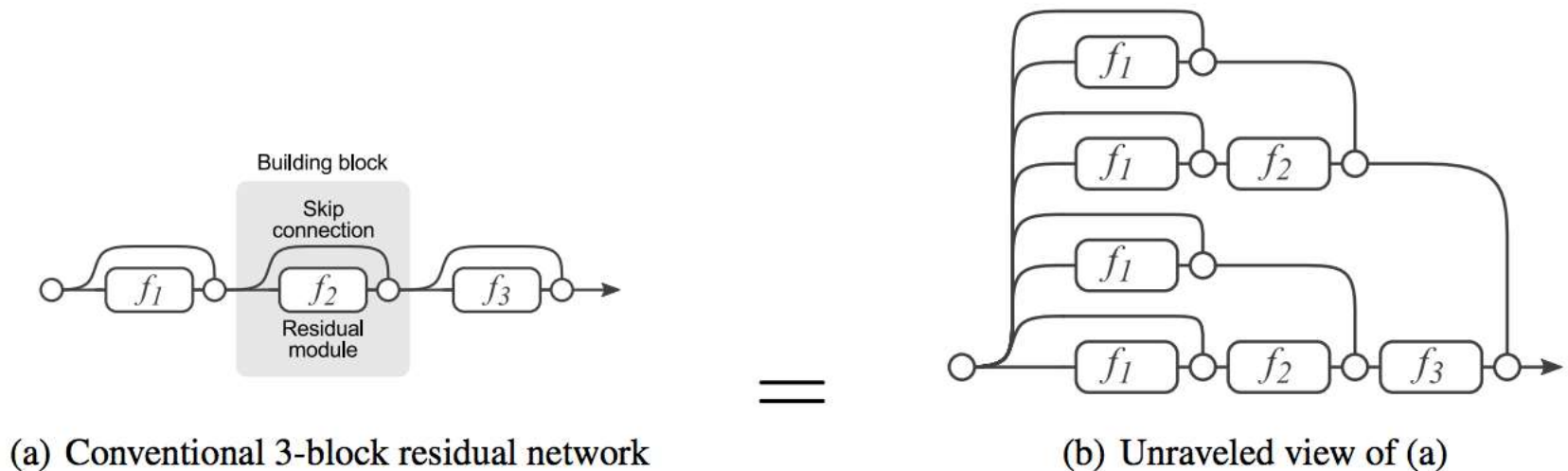
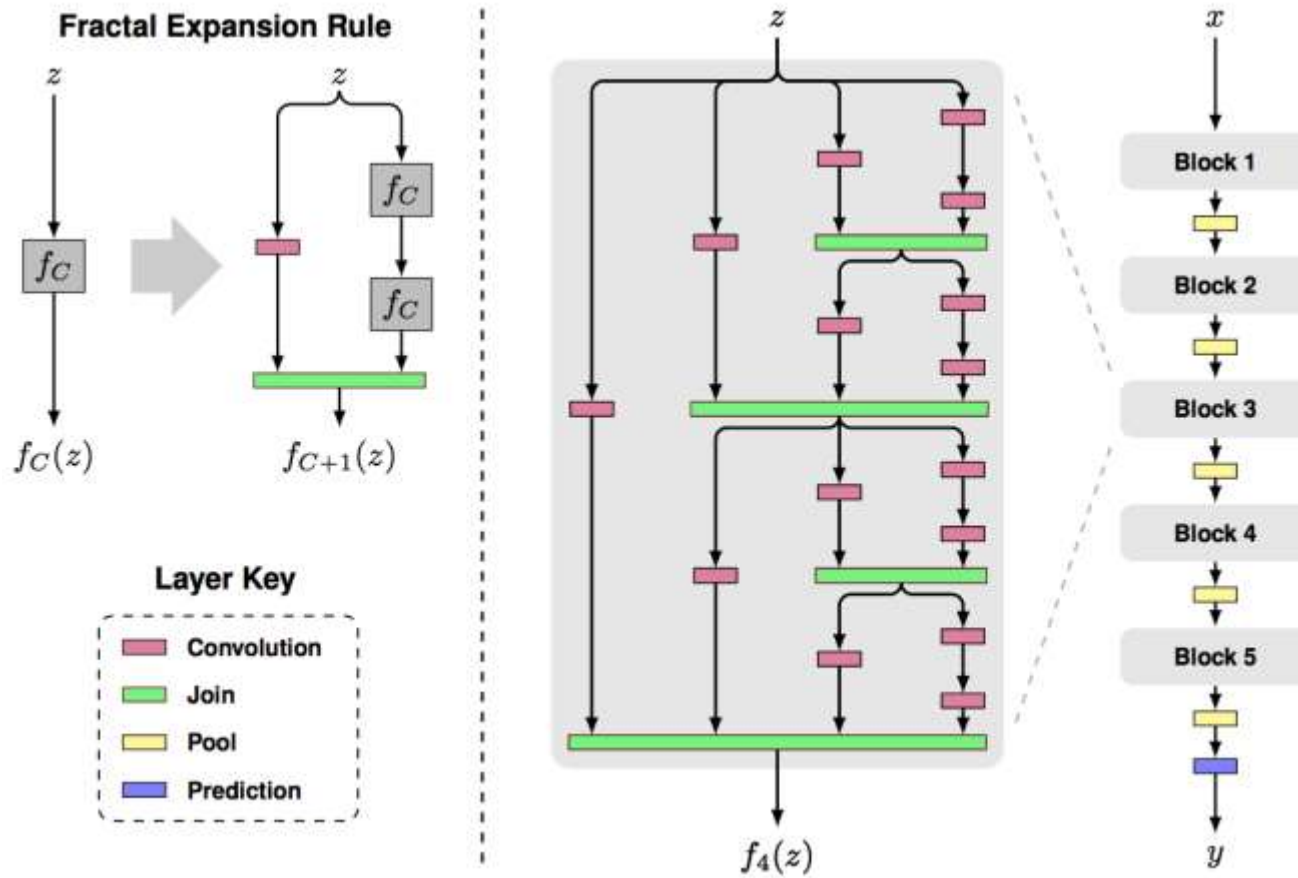


Figure 1: Residual Networks are conventionally shown as (a), which is a natural representation of Equation (1). When we expand this formulation to Equation (6), we obtain an *unraveled view* of a 3-block residual network (b). Circular nodes represent additions. From this view, it is apparent that residual networks have $O(2^n)$ implicit paths connecting input and output and that adding a block doubles the number of paths.

Beyond ResNet: FractalNet

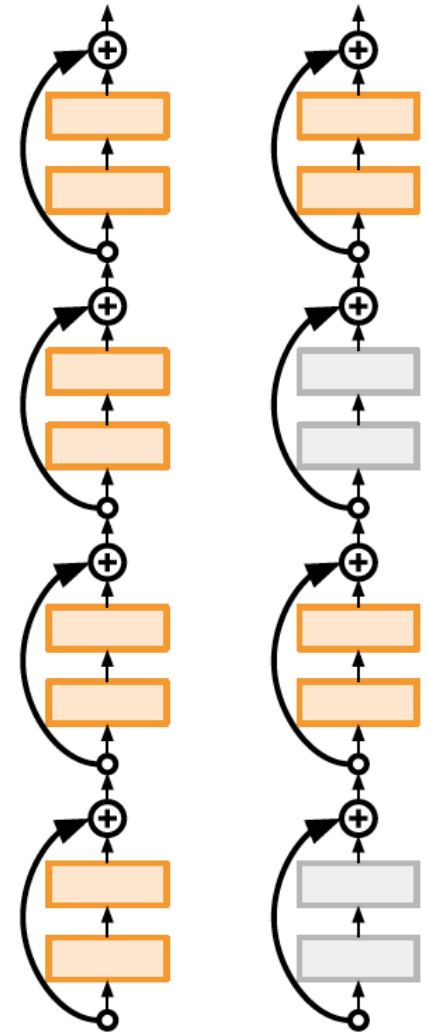
- Claim: key to good performance is not having residual connections, but having both shallow and deep paths



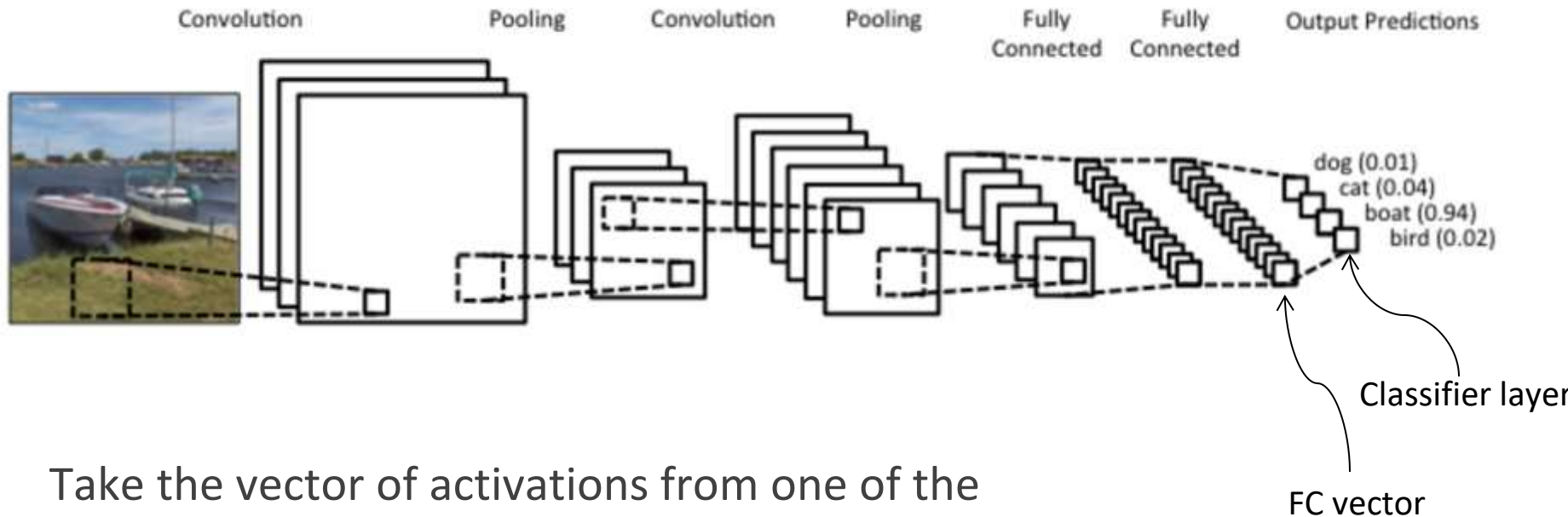
S. Larsson, M. Maire and G. Shakhnarovich, [FractalNet: Ultra-Deep Neural Networks without Residuals](#), ICLR 2017

Beyond ResNet: Stochastic depth

- At training time, in each mini-batch, randomly drop a subset of layers (bypass with identity function). At test time, use full network
- Forces the network to learn better, speeds up convergence, improves accuracy

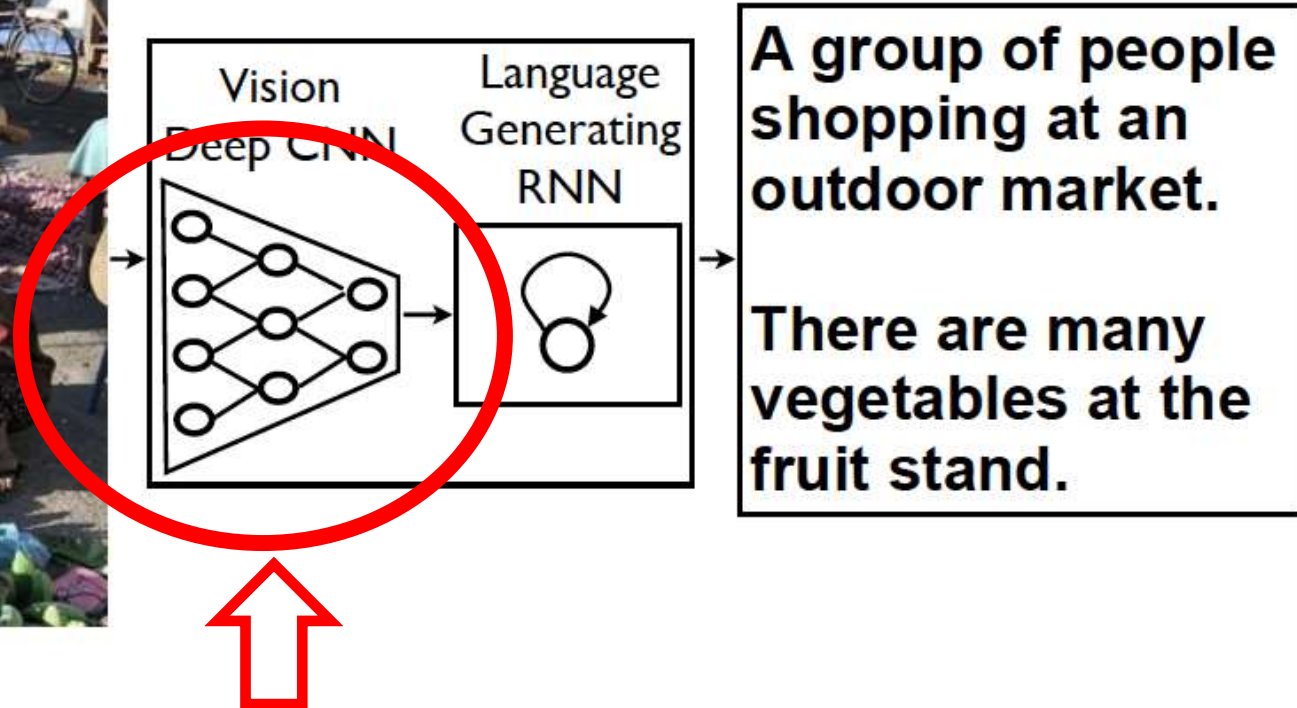


How to use a trained network for a new task?



- Take the vector of activations from one of the fully connected (FC) layers and treat it as an off-the-shelf feature
- Train a new classifier layer on top of the FC layer
- *Fine-tune* the whole network

Example: CNNs for image captioning



**FC vectors from
pre-trained network**

Summary

- CNN use convolutions to exploit grid structure
- Parameters sharing and sparse connections allow to scale to larger inputs
- Pooling introduces invariance to transformations
- Modern architectures use several tricks to increase depth and reduce computational cost

Advanced Deep Learning

3. Recurrent Neural Networks *

Prof. Jianguo Zhang
SUSTech

Learning over sequences

- A lot of data naturally comes in the form of sequences

Videos



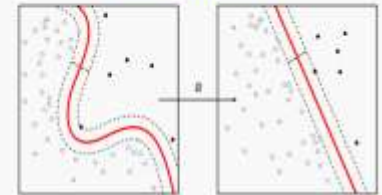
Text (and speech)

A **recurrent neural network (RNN)** is a class of **artificial neural networks** where connections between nodes form a **directed graph** along a temporal sequence. This allows it to exhibit temporal dynamic behavior. Unlike **feedforward neural networks**, RNNs can use their internal state (memory) to process sequences of inputs. This makes them applicable to tasks such as unsegmented, connected **handwriting recognition**^[1] or **speech recognition**.^{[2][3]}

The term "recurrent neural network" is used indiscriminately to refer to two broad classes of networks with a similar general structure, where one is **finite impulse** and the other is **infinite impulse**. Both classes of networks exhibit temporal **dynamic behavior**.^[4] A finite impulse recurrent network is a **directed acyclic graph** that can be unrolled and replaced with a strictly feedforward neural network, while an infinite impulse recurrent network is a **directed cyclic graph** that can not be unrolled.

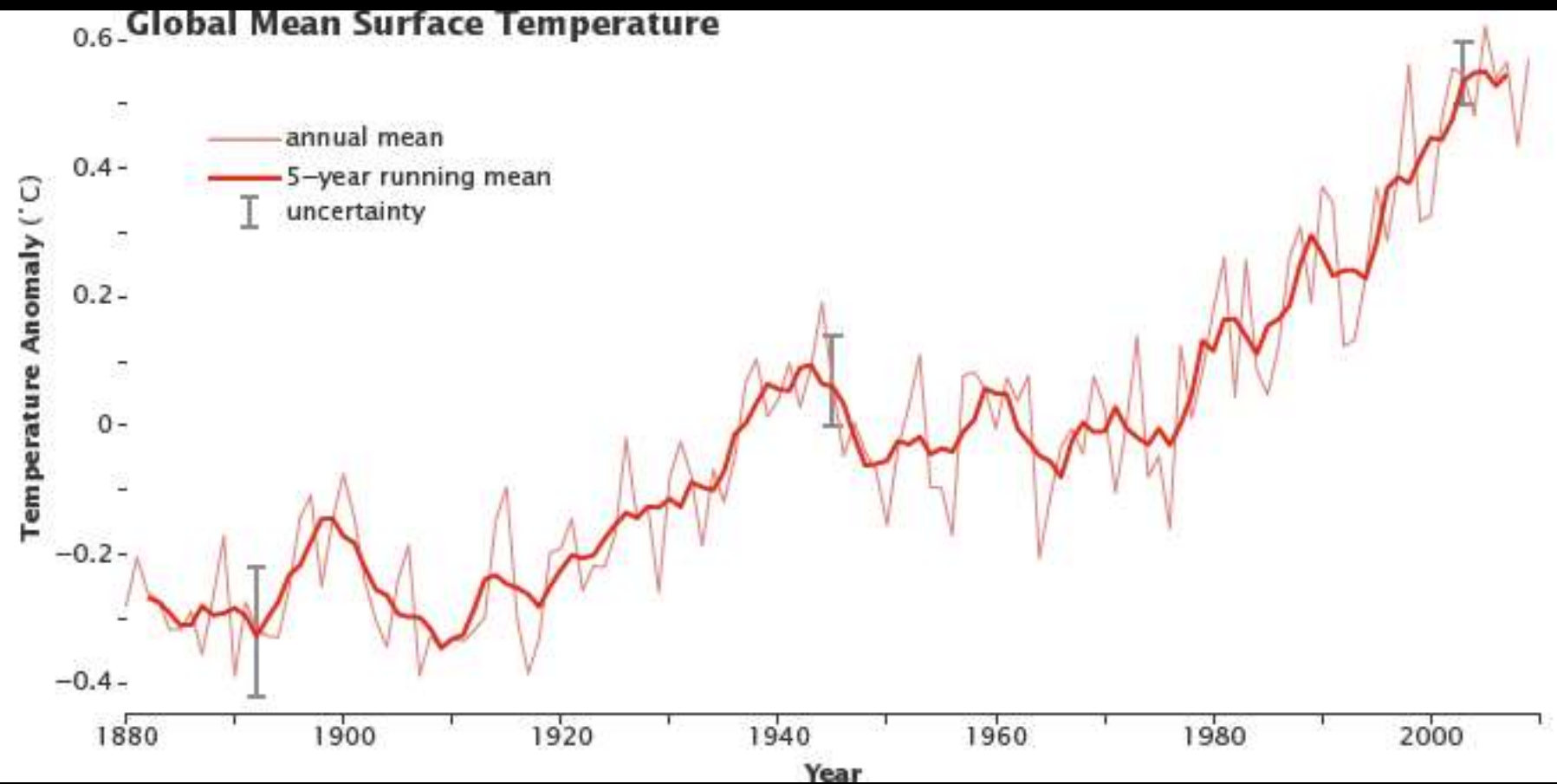
Both finite impulse and infinite impulse recurrent networks can have additional stored state, and the storage can be under direct control by the neural network. The storage can also be replaced by another network or graph, if that incorporates time delays or has feedback loops. Such controlled states are referred to as **gated state** or **gated memory**, and are part of **long short-term memory networks (LSTMs)** and **gated recurrent units**. This is also called **Feedback Neural Network**.

Machine learning and data mining



Problems	[show]
Supervised learning (classification • regression)	[show]
Clustering	[show]
Dimensionality reduction	[show]
Structured prediction	[show]

Climate measurements



Music

Ode To Joy An die Freude

Transcribed for Clarinet and Piano

L. van Beethoven

Allegro assai vivace alla Marcia ♩. = 84

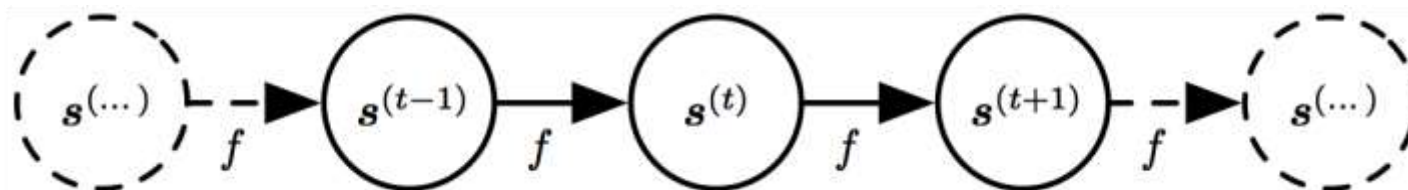
Clarinet in Bb

Piano

The image shows a musical score for the first movement of Beethoven's Ninth Symphony, 'Ode to Joy'. The score is for Clarinet in Bb and Piano. The tempo is 'Allegro assai vivace alla Marcia' with a quarter note equal to 84 beats per minute. The key signature is B-flat major (two flats). The time signature is 8/8. The Clarinet part is a single line of music with rests. The Piano part consists of two staves (treble and bass clef) with a variety of notes, rests, and dynamic markings. The dynamics include *p* (piano), *più p* (pianissimo), and *pp* (pianissimo). There are also markings for 'Ped.' (pedal) and asterisks (*) indicating specific performance instructions or ornaments. The score is presented in a clear, black-and-white format with standard musical notation.

Learning over sequences

- As with images, we need to take structure (normally time temporal dependencies) into account, but note that in sequences **future and past** are not treated “symmetrically” (but could be, e.g., bidirectional ...)
- Also, we can have very **long temporal dependencies** and **inputs/outputs** of arbitrary length, even **infinite length**



Learning over sequences

- A lot of data naturally comes in the form of sequences:
- Example:
- 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3
- How to model this sequence?
 - Modeling the temporal dependences.
- Define the states: three states: 1, 2, 3 and make some assumptions.
- Transition matrix: $f(S_t) = f(S_{t-1}, X_t)$ if we have X_t

Learning over sequences

- Transition matrix: $S_t = f(S_{t-1})$
 - 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3
 - 3
- However, it can't model the sequence below
(Example (noisy):
- 1.1, 2.2, 3.1, 1.3, 2.2, 3.3, 1.4, 2.4, 3.3, 1.1, 2.0, 3.1, 1.2, 2.1, 3.0, 1.1, 2.3, 3.4,
 - we need to use $S_t = f(S_{t-1}, X_t)$

Learning over sequences

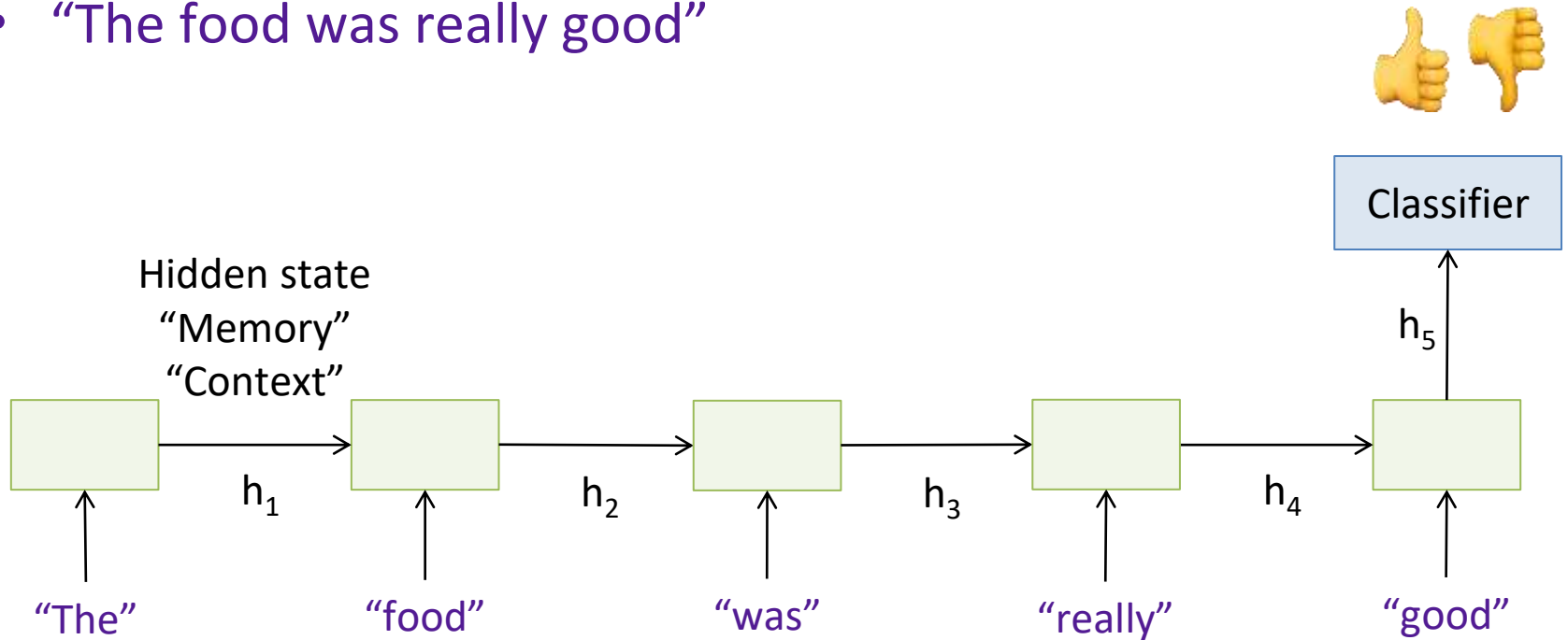
- Consider the word 'mountain'
- Given the string 'mountai', there is strong probability that the next character will be 'n'
- This is precisely what we want our neural network to be able to model
- More in general, we need to somehow **model context and memory**

Text classification: Examples

- **Sentiment classification:** classify a restaurant or movie or product review as positive or negative
 - “The food was really good”
 - “The vacuum cleaner broke within two weeks”
 - “The movie had slow parts, but overall was worth watching”
- What feature representation or predictor structure can we use for this problem?

Sentiment classification

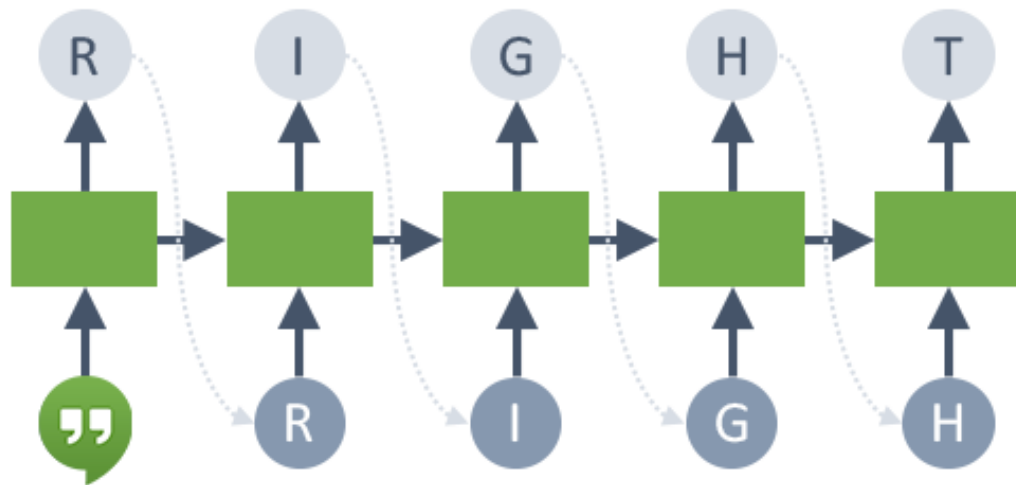
- “The food was really good”



Recurrent Neural Network (RNN)

Language Modeling

- Character RNN



Input representation

- In the following we assume our inputs are either characters (e.g., the network's task is to predict the next one) or words (e.g., the network's task is to predict the next word)
- In general, we will have a sequence of symbols in input that can be drawn from a larger vocabulary

Input representation

- We start with a one-hot vector representation

Rome = $[1, 0, 0, 0, 0, 0, \dots, 0]$

Paris = $[0, 1, 0, 0, 0, 0, \dots, 0]$

Italy = $[0, 0, 1, 0, 0, 0, \dots, 0]$

France = $[0, 0, 0, 1, 0, 0, \dots, 0]$

Input representation

- We start with a one-hot vector representation

Rome = $[1, 0, 0, 0, 0, 0, \dots, 0]$

Paris = $[0, 1, 0, 0, 0, 0, \dots, 0]$

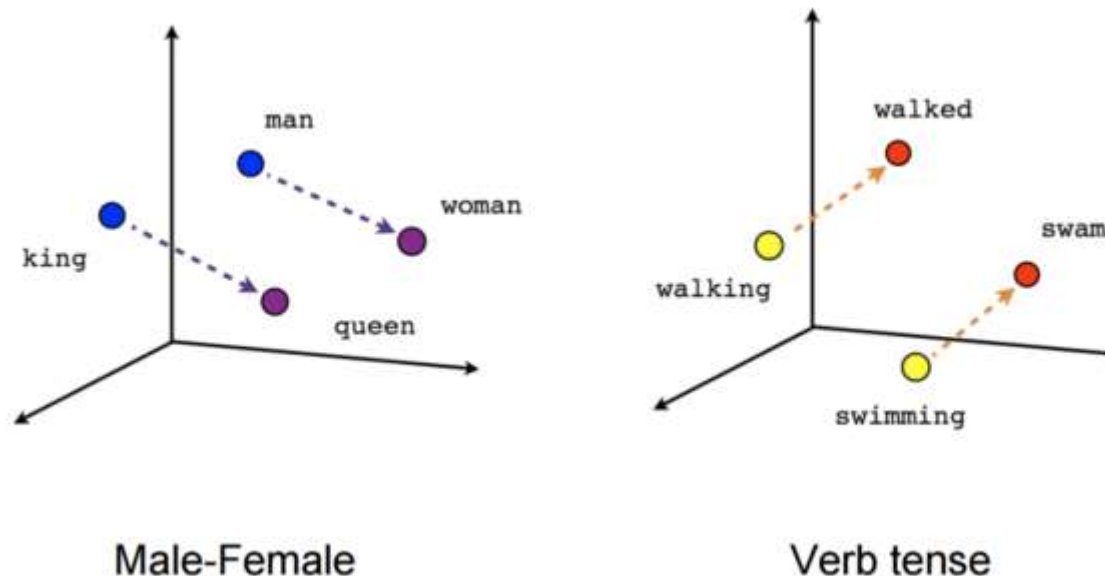
Italy = $[0, 0, 1, 0, 0, 0, \dots, 0]$

France = $[0, 0, 0, 1, 0, 0, \dots, 0]$

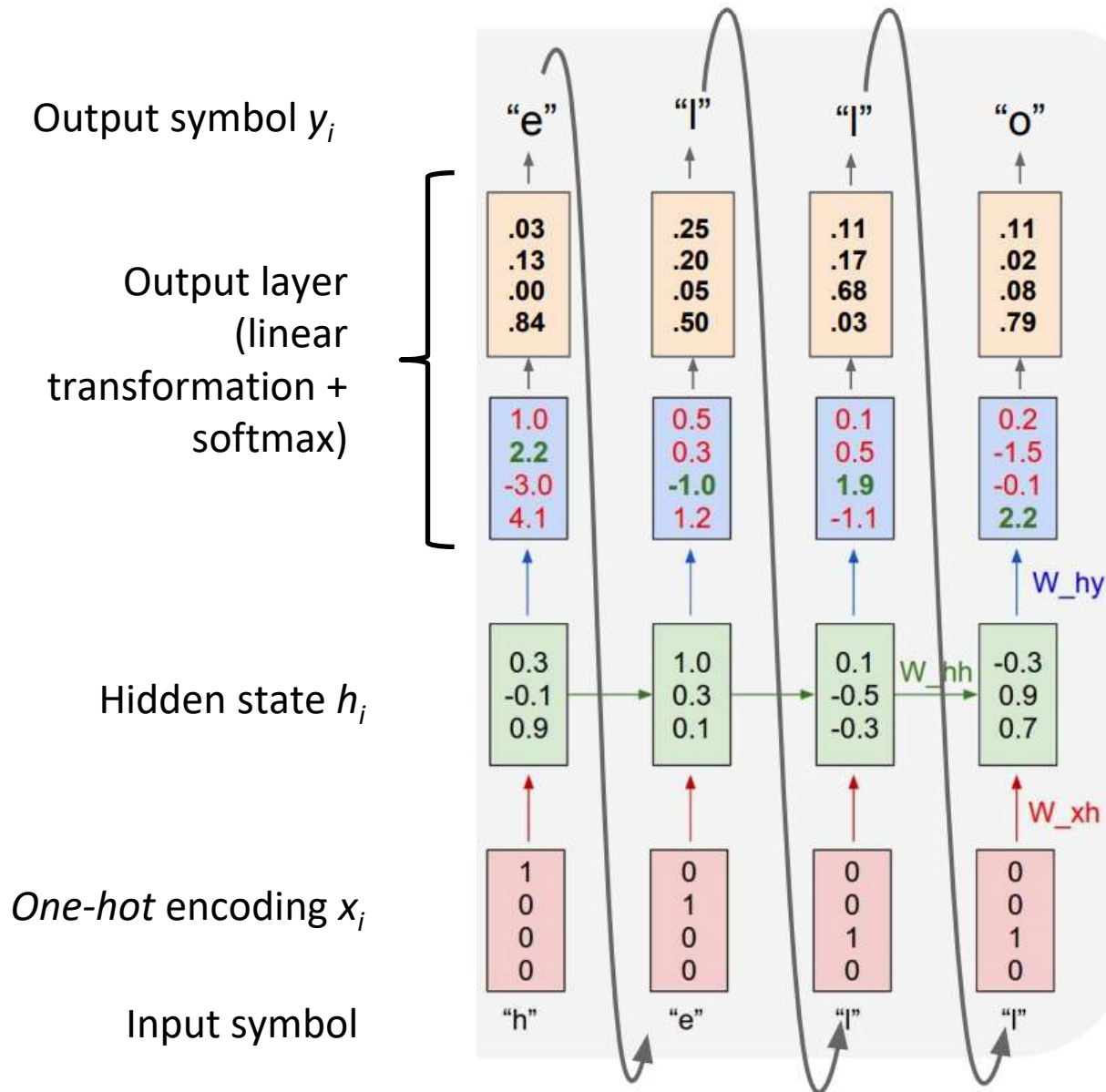
Note: we can do the same if our inputs are characters and one-hot encode them

Input representation

- We start with a one-hot vector representation
- If we're dealing with words, after the one-hot vector, we apply an embedding (e.g., Word2Vec)

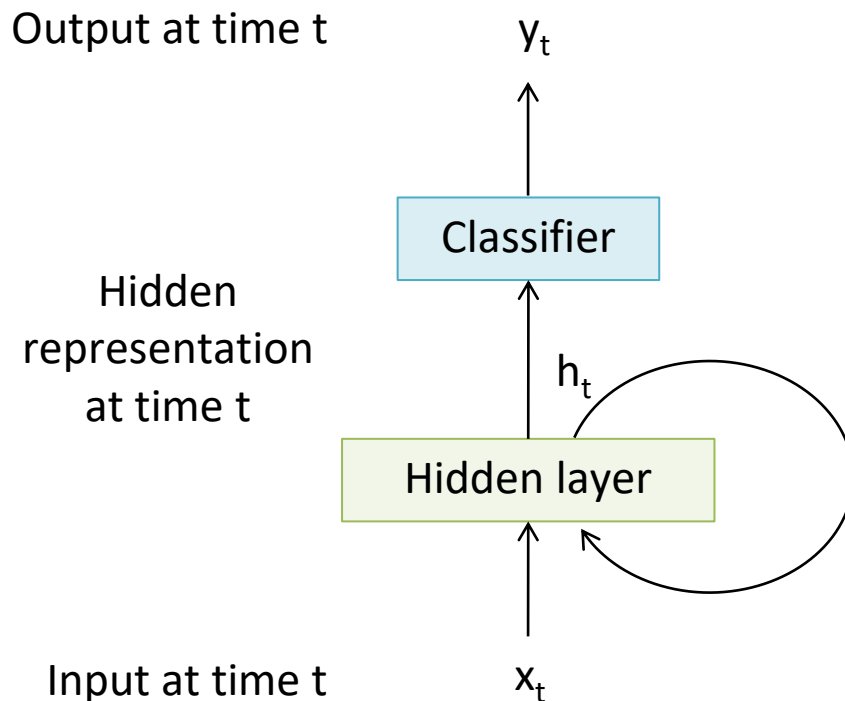


Character RNN



$$\begin{aligned}
 & p(y_1, y_2, \dots, y_n) \\
 &= \prod_{i=1}^n p(y_i | y_1, \dots, y_{i-1}) \\
 &\approx \prod_{i=1}^n P_W(y_i | h_i)
 \end{aligned}$$

Recurrent Neural Network (RNN)

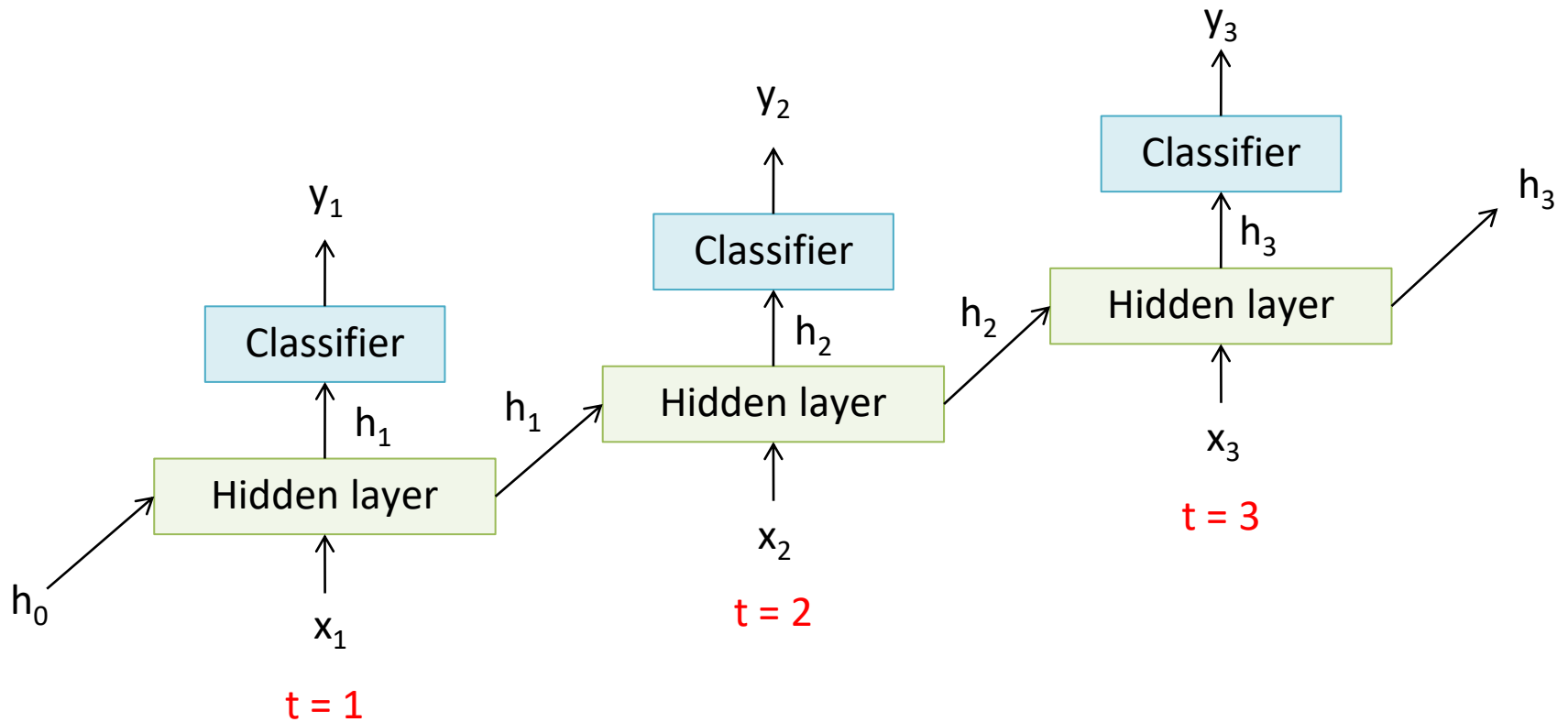


Recurrence:

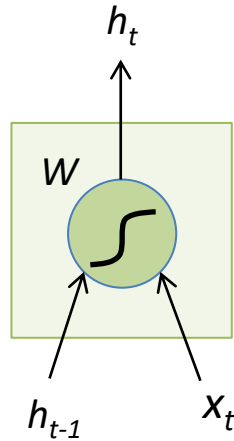
$$h_t = f_W(x_t, h_{t-1})$$

new state function of W input at time t old state

Unrolling the RNN

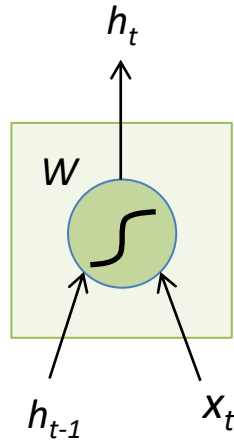


Vanilla RNN Cell

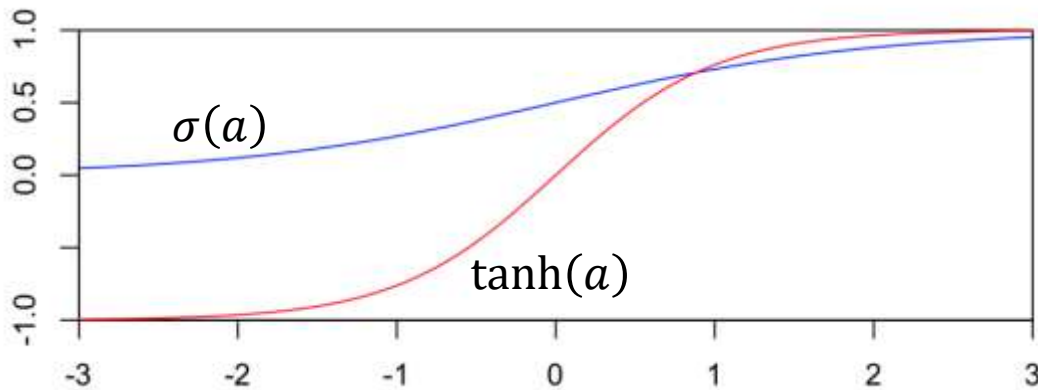


$$\begin{aligned} h_t &= f_W(x_t, h_{t-1}) \\ &= \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix} \end{aligned}$$

Vanilla RNN Cell

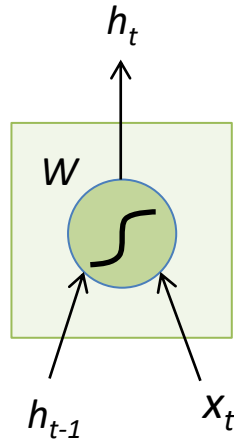


$$\begin{aligned} h_t &= f_W(x_t, h_{t-1}) \\ &= \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix} \end{aligned}$$

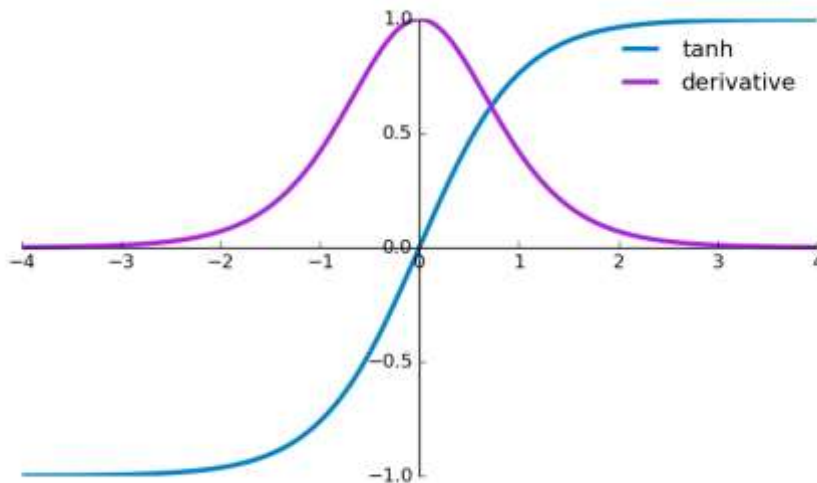


$$\begin{aligned} \tanh(a) &= \frac{e^a - e^{-a}}{e^a + e^{-a}} \\ &= 2\sigma(2a) - 1 \end{aligned}$$

Vanilla RNN Cell

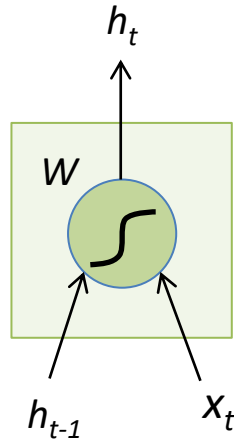


$$\begin{aligned} h_t &= f_W(x_t, h_{t-1}) \\ &= \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix} \end{aligned}$$

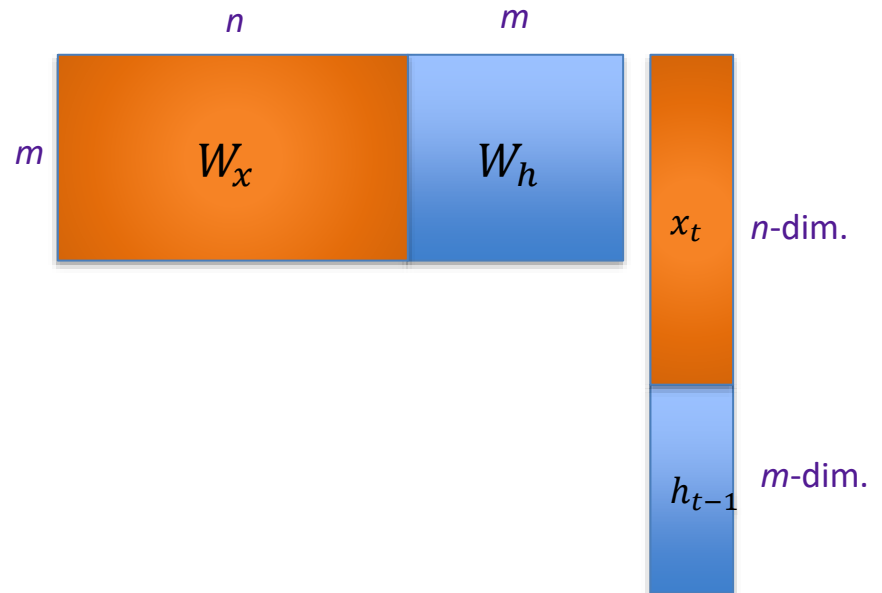


$$\frac{d}{da} \tanh(a) = 1 - \tanh^2(a)$$

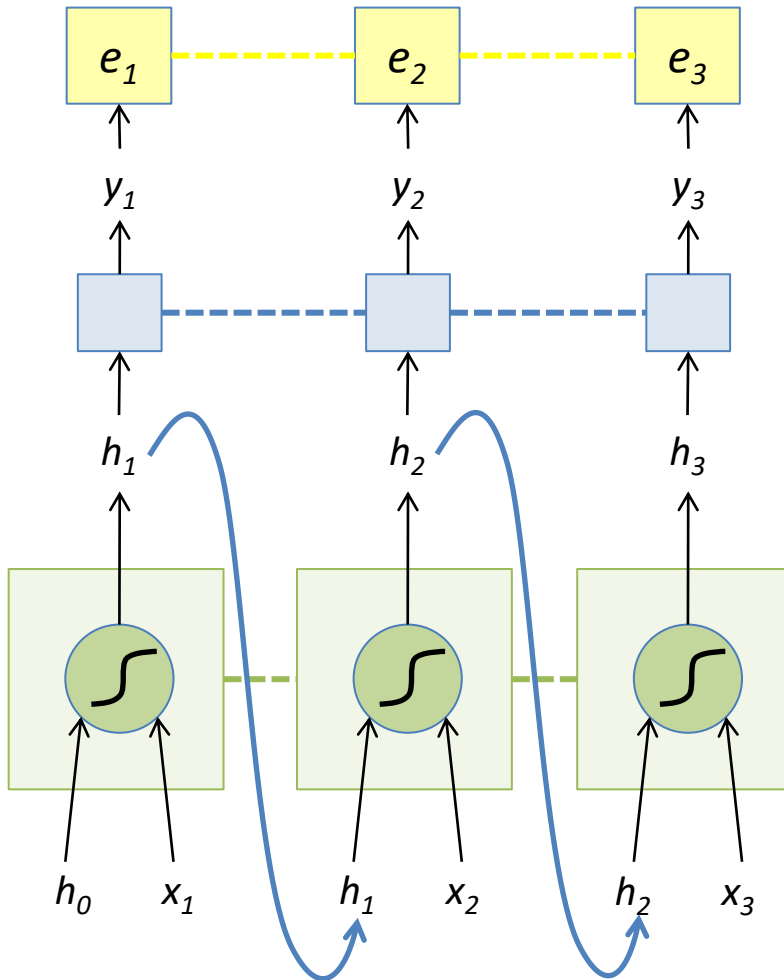
Vanilla RNN Cell



$$\begin{aligned} h_t &= f_W(x_t, h_{t-1}) \\ &= \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix} \\ &= \tanh(W_x x_t + W_h h_{t-1}) \end{aligned}$$



RNN Forward Pass



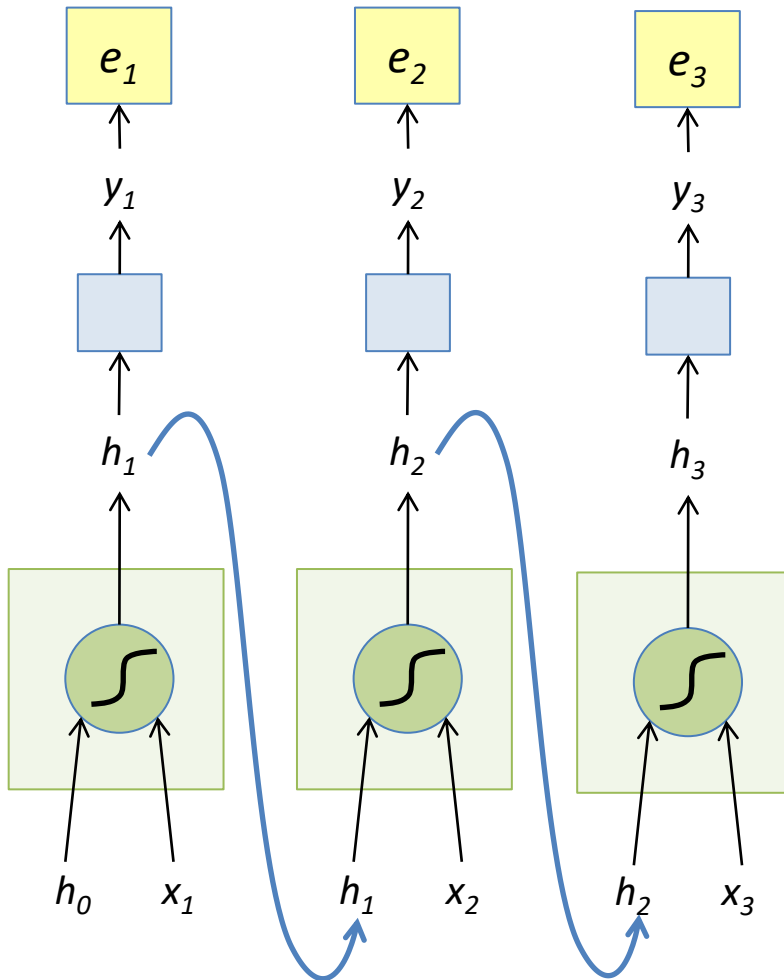
$$e_t = -\log(y_t(GT_t))$$

$$y_t = \text{softmax}(W_y h_t)$$

$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

----- shared weights

Unfolded RNN Forward Pass

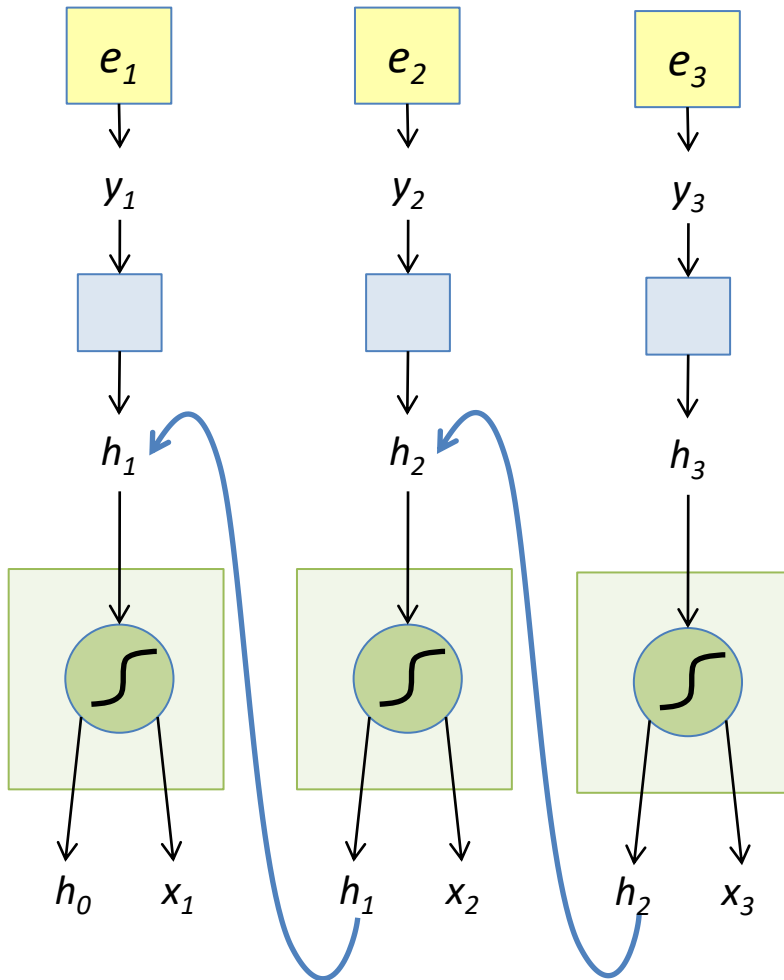


$$e_t = -\log(y_t(GT_t))$$

$$y_t = \text{softmax}(W_y h_t)$$

$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

Unfolded RNN Backward Pass



$$e_t = -\log(y_t(GT_t))$$

$$y_t = \text{softmax}(W_y h_t)$$

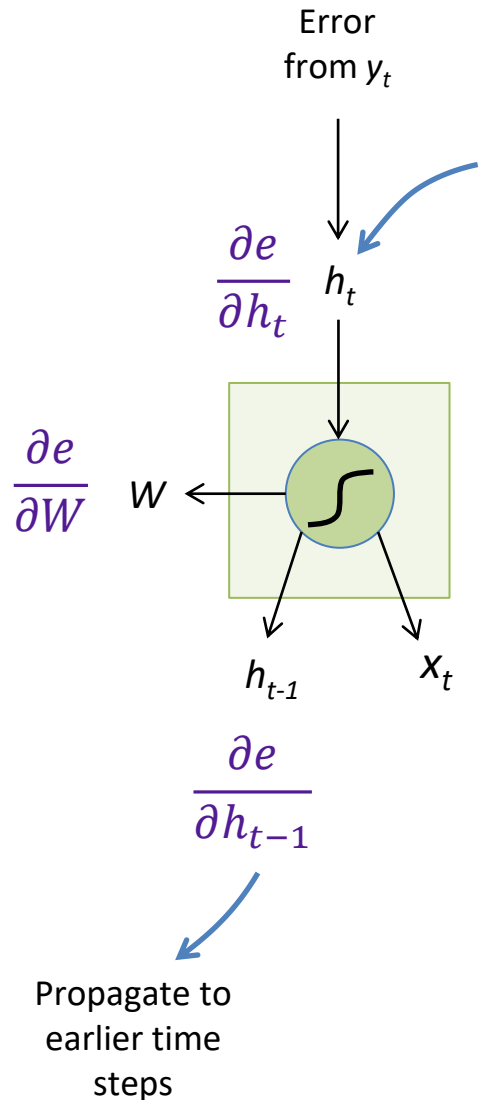
$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

Backpropagation Through Time (BPTT)

- Most common method used to train RNNs
- The unfolded network (used during forward pass) is treated as one big feed-forward network that accepts the whole time series as input
- The weight updates are computed for each copy in the unfolded network, then summed (or averaged) and applied to the RNN weights
- In practice, *truncated* BPTT is used: run the RNN forward k_1 time steps, propagate backward for k_2 time steps

<https://machinelearningmastery.com/gentle-introduction-backpropagation-time/>
http://www.cs.utoronto.ca/~ilya/pubs/ilya_sutskever_phd_thesis.pdf

RNN Backward Pass



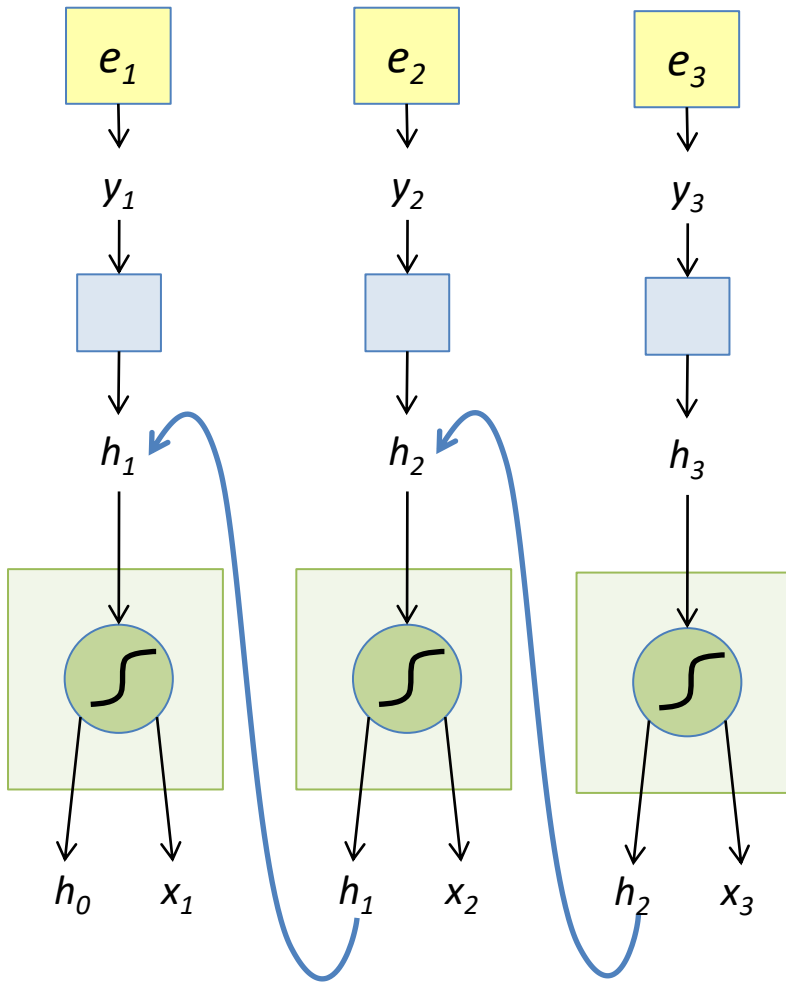
$$h_t = \tanh(W_x x_t + W_h h_{t-1})$$

$$\frac{\partial e}{\partial W_h} = \frac{\partial e}{\partial h_t} \odot (1 - \tanh^2(W_x x_t + W_h h_{t-1})) h_{t-1}^T$$

$$\frac{\partial e}{\partial W_x} = \frac{\partial e}{\partial h_t} \odot (1 - \tanh^2(W_x x_t + W_h h_{t-1})) x_t^T$$

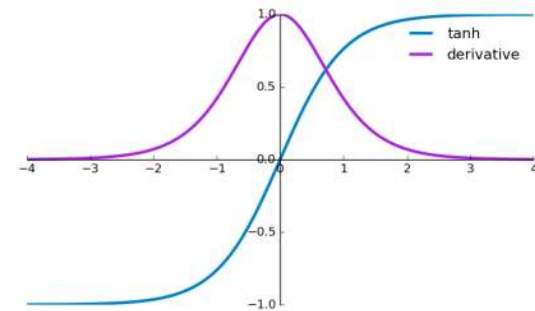
$$\frac{\partial e}{\partial h_{t-1}} = W_h^T (1 - \tanh^2(W_x x_t + W_h h_{t-1})) \odot \frac{\partial e}{\partial h_t}$$

RNN Backward Pass



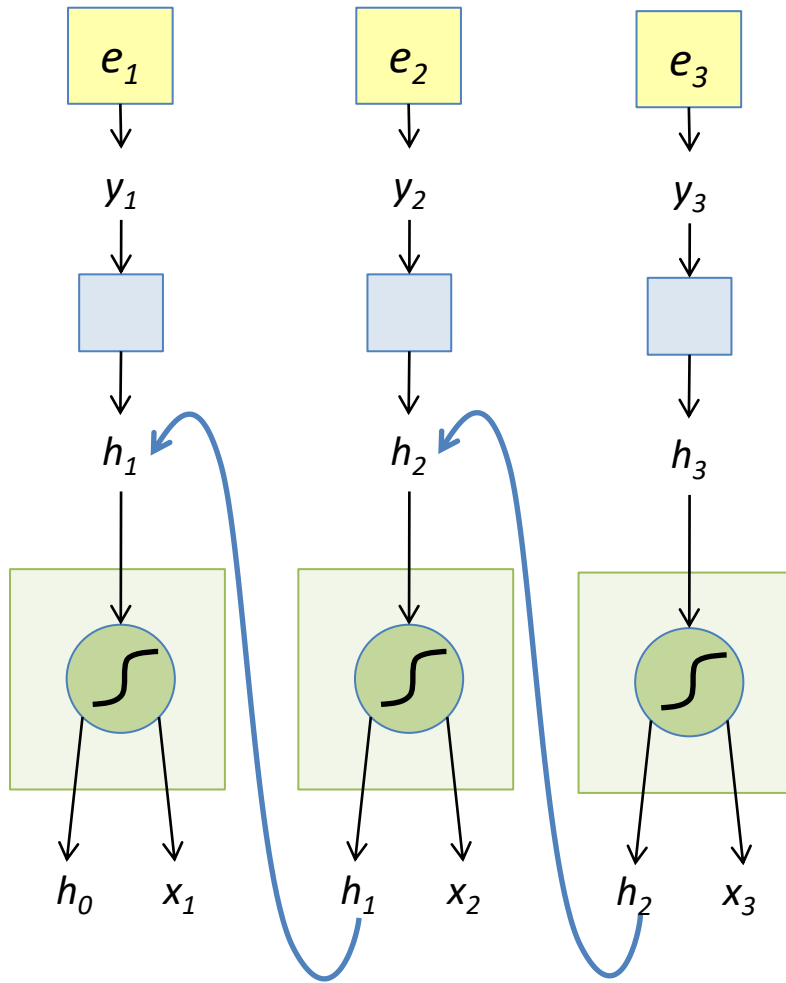
$$\frac{\partial e}{\partial h_{t-1}} = W_h^T (1 - \tanh^2(W_x x_t + W_h h_{t-1})) \odot \frac{\partial e}{\partial h_t}$$

Large tanh activations will give small gradients



Consider $\frac{\partial e_n}{\partial h_k}$ for $k \ll n$

RNN Backward Pass



$$\frac{\partial e}{\partial h_{t-1}} = W_h^T (1 - \tanh^2(W_x x_t + W_h h_{t-1})) \odot \frac{\partial e}{\partial h_t}$$

Gradients will vanish if
largest singular value of
 W_h is less than 1

Consider $\frac{\partial e_n}{\partial h_k}$ for $k \ll n$

Backpropagation through time

- Backpropagation is done similarly to MLP/CNN
- However the layers now correspond to different steps/times, so we call it **backpropagation through time** (section 10.2.2. in the book)
- As with very deep networks we need to multiply together many different gradients
- For long sequences, this is a problem
 - Vanishing/exploding gradients